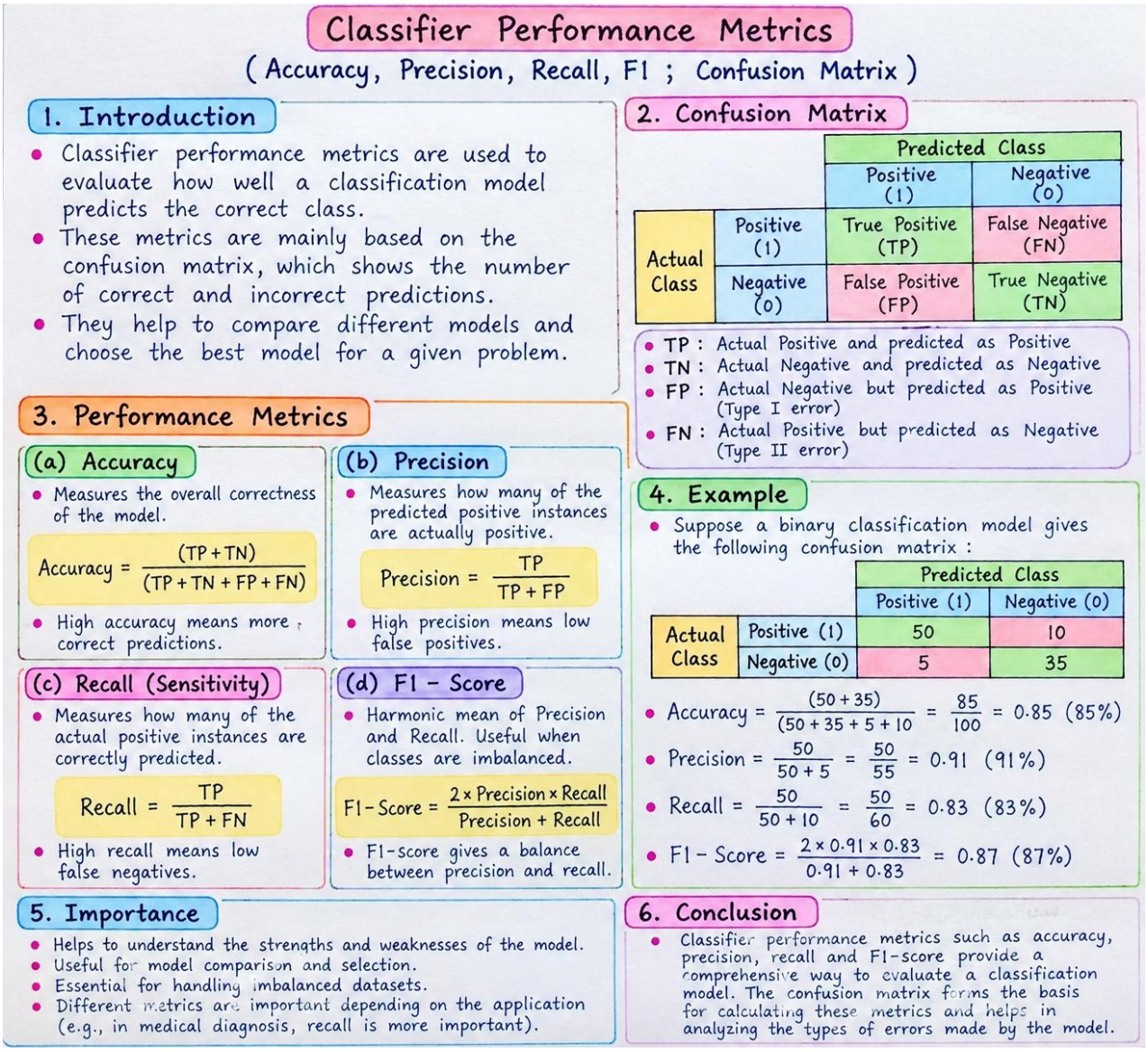
Q1. Classifier performance metrics (accuracy, precision, recall, F1; confusion matrix

ANS



Introduction

Classifier performance metrics are used to evaluate how effectively a classification model predicts the correct class of data. These metrics help us measure the accuracy of a model and identify its errors.

The main classifier performance metrics are:

1. Accuracy

2. Precision
3. Recall
4. F1-Score
5. Confusion Matrix

2. Confusion Matrix

A confusion matrix is a table used to evaluate the performance of a classification model by comparing actual values with predicted values.

Actual / Predicted	Positive	Negative
Positive	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP)	True Negative (TN)

The four terms are explained as follows:

- True Positive (TP): The model correctly predicts the positive class.
- True Negative (TN): The model correctly predicts the negative class.
- False Positive (FP): The model predicts positive, but the actual class is negative.
- False Negative (FN): The model predicts negative, but the actual class is positive.

The confusion matrix is the basis for calculating accuracy, precision, recall and F1-score.

3. Accuracy

Accuracy measures the overall correctness of a classification model. It represents the proportion of total predictions that are correct.

Formula:

$$\text{Accuracy}=\frac{TP+TN}{TP+TN+FP+FN}$$

Advantages:

- Easy to understand and calculate.
- Useful when the dataset has a balanced number of classes.

Limitation: Accuracy may be misleading when the dataset is highly imbalanced.

4. Precision

Precision measures how many of the instances predicted as positive are actually positive.

Formula:

$$\text{Precision}=\frac{TP}{TP+FP}$$

Advantages:

- Helps reduce false-positive predictions.
- Useful in spam detection, where incorrectly classifying a genuine email as spam should be avoided.

A high precision value indicates that most positive predictions made by the model are correct.

5. Recall (Sensitivity)

Recall measures how many actual positive instances are correctly identified by the model.

Formula:

$$\text{Recall}=\frac{TP}{TP+FN}$$

Advantages:

- Helps reduce false-negative predictions.
- Important in medical diagnosis, where missing a disease case can be dangerous.

A high recall value indicates that the model identifies most of the actual positive cases.

6. F1-Score

F1-score is the harmonic mean of precision and recall. It provides a balance between these two metrics.

Formula:

$$F1=2\times\frac{Precision\times Recall}{Precision+Recall}$$

Advantages:

- Balances precision and recall.
- Useful for imbalanced datasets.
- Provides a single score when both false positives and false negatives matter.

A higher F1-score generally indicates a better balance between precision and recall.

7. Numerical Example

Suppose a classification model produces the following results:

- True Positive (TP) = 50
- True Negative (TN) = 35
- False Positive (FP) = 5
- False Negative (FN) = 10

1. Accuracy

$$=\frac{50+35}{50+35+5+10} =\frac{85}{100}=85\%$$

2. Precision

$$=\frac{50}{50+5} =\frac{50}{55}=90.91\%$$

3. Recall

$$=\frac{50}{50+10} =\frac{50}{60}=83.33\%$$

4. F1-Score

$$=2\times\frac{0.9091\times0.8333}{0.9091+0.8333}$$

$$\approx 0.8696=86.96\%$$

Therefore, the model has 85% accuracy, 90.91% precision, 83.33% recall and 86.96% F1-score.

8. Applications

1. Medical diagnosis: Recall helps identify patients who have a disease.
2. Spam detection: Precision helps avoid marking genuine emails as spam.
3. Fraud detection: Recall helps identify fraudulent transactions.
4. Image classification: Accuracy measures the overall correctness of predictions.
5. Imbalanced datasets: F1-score helps evaluate performance when both precision and recall are important.

Q2. Mamdani fuzzy controller: washing-machine wash time

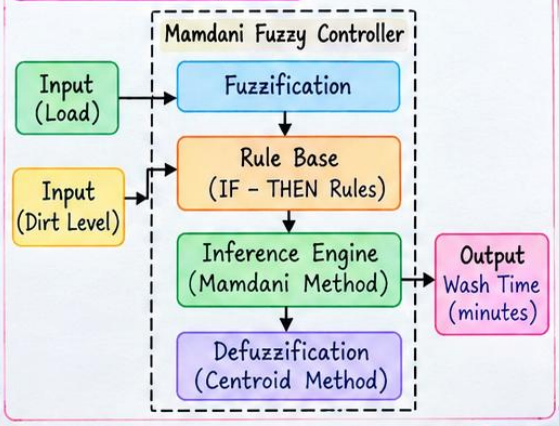
ANS

Mamdani Fuzzy Controller : Washing-Machine Wash Time

1. Introduction

- Fuzzy logic is used to handle uncertain and imprecise information similar to human reasoning.
- Mamdani fuzzy controller is the most commonly used fuzzy inference system.
- In a washing machine, the wash time depends on the amount of load and how dirty the clothes are.
- Mamdani fuzzy controller is used to determine the optimal wash time automatically.

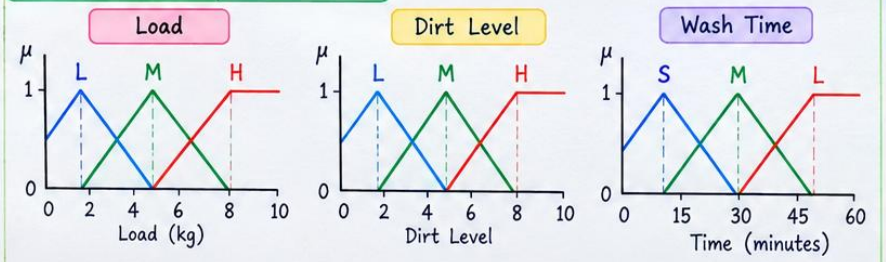
2. Block Diagram



3. Input and Output Variables

Variable	Linguistic Terms	Range
Load (kg)	Low (L) Medium (M) High (H)	0 – 10 kg
Dirt Level	Low (L) Medium (M) High (H)	0 – 10 (level)
Wash Time (minutes)	Short (S) Medium (M) Long (L)	0 – 60 min

4. Membership Functions



5. Rule Base (Mamdani)

Rule No.	Load	Dirt Level	Wash Time
1	Low	Low	Short
2	Low	Medium	Short
3	Low	High	Medium
4	Medium	Low	Short
5	Medium	Medium	Medium
6	Medium	High	Long
7	High	Low	Medium
8	High	Medium	Long
9	High	High	Long

6. Working of Mamdani Fuzzy Controller

1. **Fuzzification** : Convert crisp inputs (load and dirt level) into fuzzy values using membership functions.
2. **Rule Evaluation** : Apply IF-THEN fuzzy rules using AND operator (min).
3. **Inference Engine** : Combine all rule outputs to get aggregated fuzzy output (max).
4. **Defuzzification** : Convert aggregated fuzzy output into a single crisp value using Centroid method.
5. **Output** : Gives the wash time in minutes.

7. Example

Consider Load = 7 kg and Dirt Level = 8.

- From membership functions:
  Load → High (0.8), Medium (0.2)
  Dirt Level → High (0.7), Medium (0.3)
- Relevant rules (fired):
  R5: M ∧ M → Medium ($\mu = \min(0.2, 0.3) = 0.2$)
  R6: M ∧ H → Long ($\mu = \min(0.2, 0.7) = 0.2$)
  R8: H ∧ M → Long ($\mu = \min(0.8, 0.3) = 0.3$)
  R9: H ∧ H → Long ($\mu = \min(0.8, 0.7) = 0.7$)
- Aggregated output : Mainly 'Long' with highest membership 0.7.
- After defuzzification (Centroid method), Wash Time ≈ 45 minutes.

8. Advantages and Applications

Advantages

- Handles imprecise and uncertain data.
- Simple and easy to implement.
- Works like human reasoning.
- Provides smooth and flexible control.

Applications

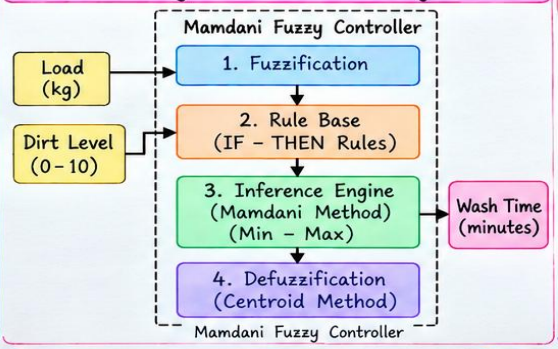
- Automatic washing machines (wash time control).
- Home appliances (AC, refrigerator).
- Industrial process control.
- Robotics and intelligent systems.

Mamdani Fuzzy Controller : Washing – Machine Wash Time

1. Introduction

- Fuzzy logic is used to handle uncertain and imprecise information similar to human reasoning.
- Mamdani fuzzy controller is the most popular fuzzy inference system.
- In a washing machine, the wash time depends on the amount of load and how dirty the clothes are.
- Mamdani fuzzy controller uses linguistic rules to determine the optimal wash time automatically.

2. Block Diagram of Mamdani Fuzzy Controller

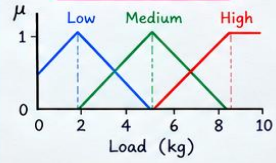


3. Input and Output Variables

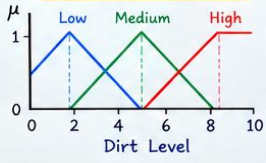
Variable	Linguistic Terms	Range
Load (kg)	Low (L) Medium (M) High (H)	0 – 10 kg
Dirt Level	Low (L) Medium (M) High (H)	0 – 10 (level)
Wash Time (minutes)	Short (S) Medium (M) Long (L)	0 – 60 min

4. Membership Functions

(a) Load (kg)



(b) Dirt Level (0 – 10)



(c) Wash Time (minutes)



5. Rule Base (IF – THEN Rules)

Rule No.	Load	Dirt Level	Wash Time
1	Low	Low	Short
2	Low	Medium	Short
3	Low	High	Medium
4	Medium	Low	Short
5	Medium	Medium	Medium
6	Medium	High	Long
7	High	Low	Medium
8	High	Medium	Long
9	High	High	Long

6. Working Steps of Mamdani Fuzzy Controller

1. **Fuzzification :** Convert crisp inputs (load and dirt level) into fuzzy values using membership functions.
2. **Rule Evaluation :** Apply IF–THEN rules using AND operator (min) to get rule outputs.
3. **Inference Engine :** Combine all rule outputs using Max operator (Mamdani method).
4. **Defuzzification :** Convert aggregated fuzzy output into single crisp value using Centroid method.
5. **Output :** Gives the wash time in minutes.

7. Example

Consider : Load = 7 kg , Dirt Level = 8

Step 1 : Fuzzification

- From membership functions:
  - Load = High (0.8), Medium (0.2)
  - Dirt Level = High (0.7), Medium (0.3)

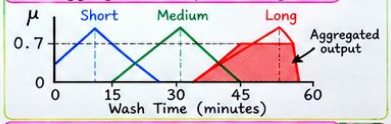
Step 2 : Rule Evaluation (min operator)

- R6: Medium & High → Long (min(0.2,0.7) = 0.2)
- R8: High & Medium → Long (min(0.8,0.3) = 0.3)
- R9: High & High → Long (min(0.8,0.7) = 0.7)
- Other rules give smaller values.

Step 3 : Aggregation (max operator)

- Aggregated output = Long (maximum = 0.7)

8. Aggregated Output (Fuzzy Set)



9. Defuzzification (Centroid Method)

Centroid of the aggregated area gives:

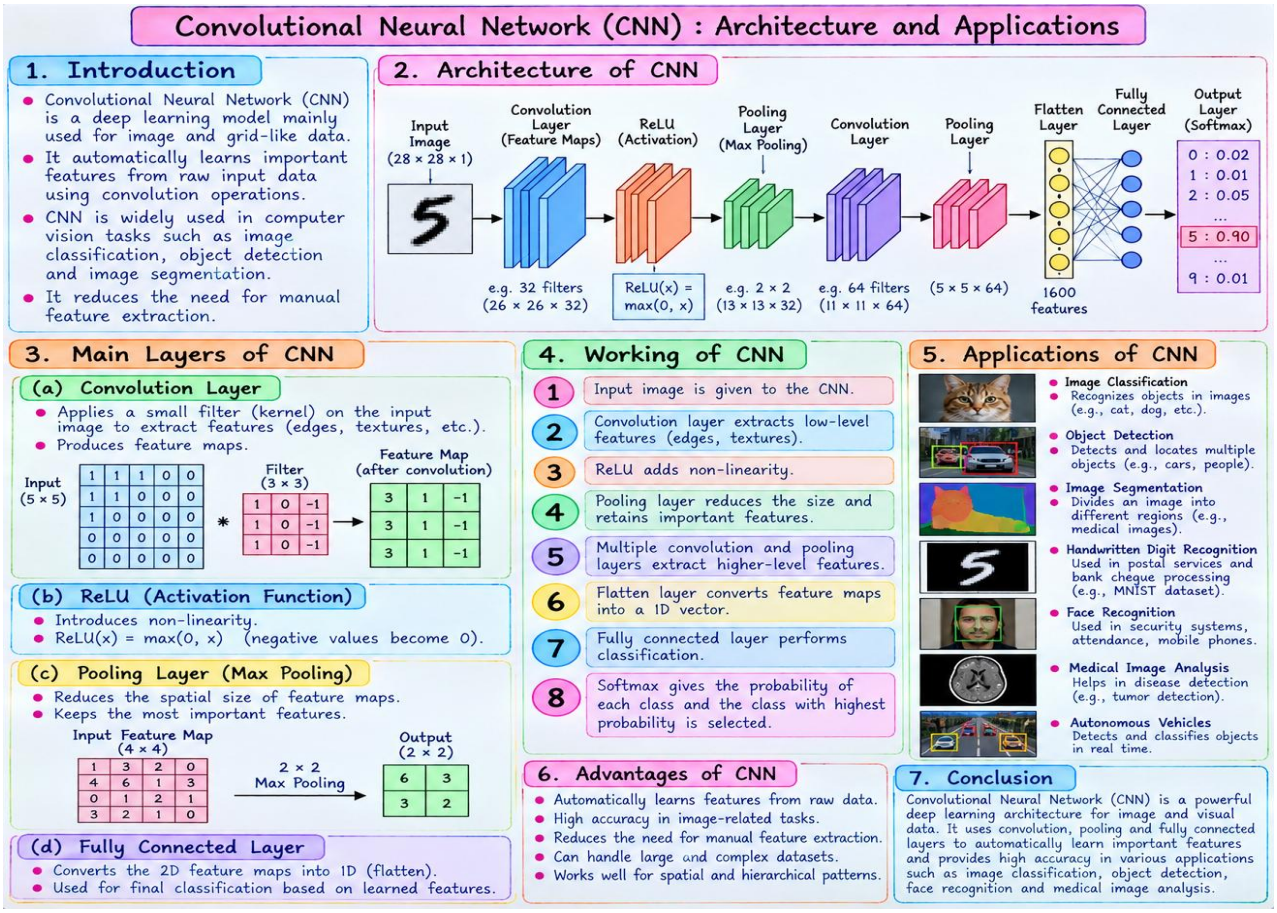
Wash Time ≈ 45 minutes

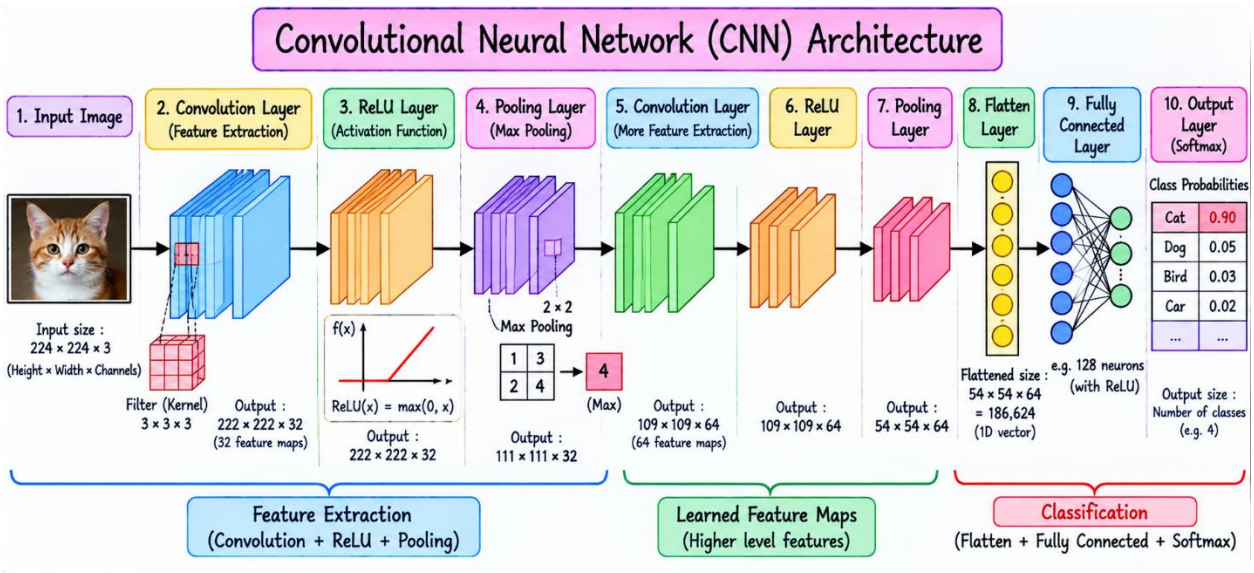
10. Conclusion

- Mamdani fuzzy controller provides a simple and effective way to determine washing machine wash time.
- It handles uncertain and imprecise inputs and gives a human-like decision using fuzzy rules.

Q3. CNN architecture and applications

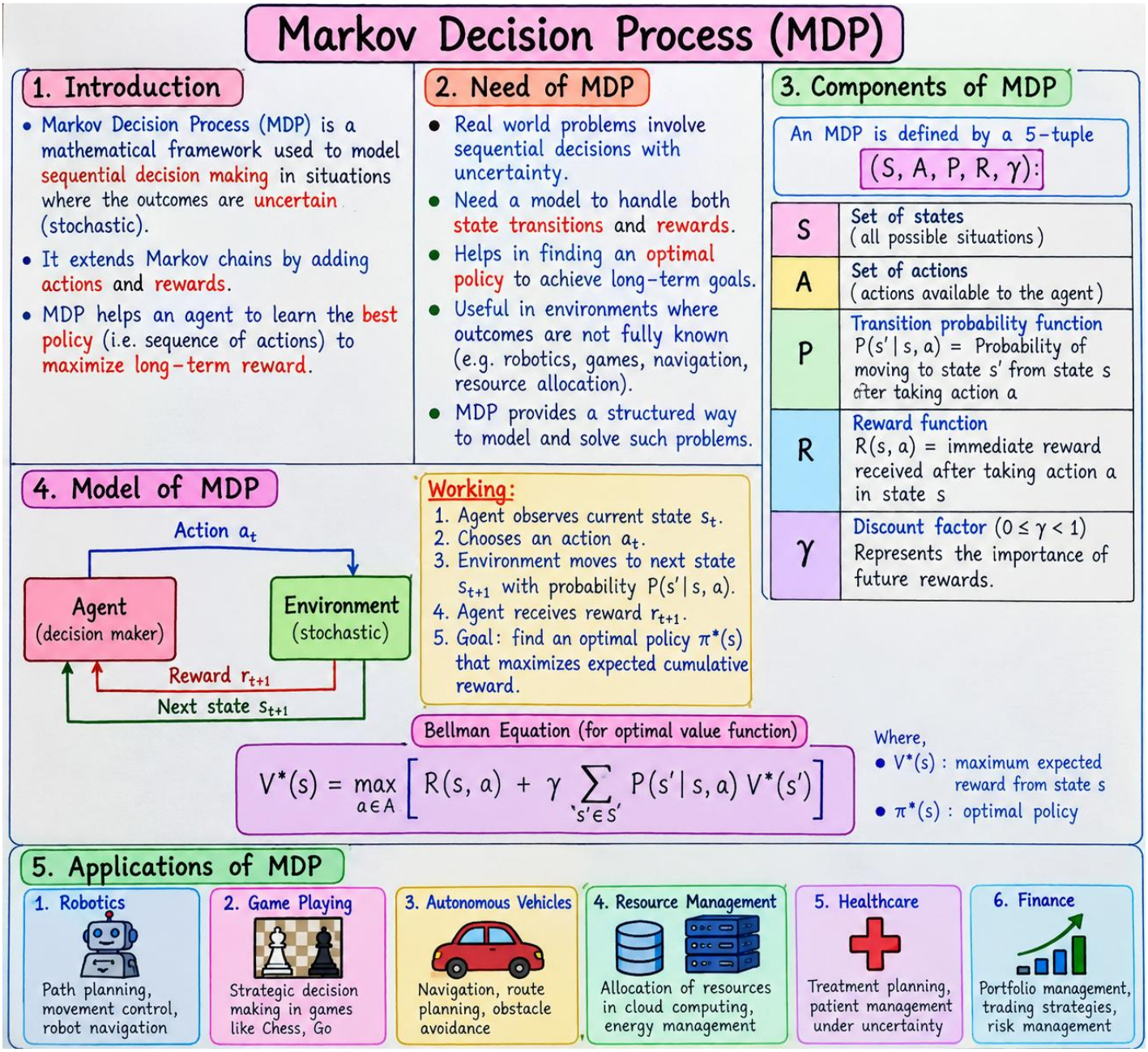
ANS





Q4. Markov Decision Process (MDP): need, components, model, applications

ANS



Markov Decision Process (MDP)

1. Introduction

- Markov Decision Process (MDP) is a mathematical framework for sequential decision making in situations where outcomes are uncertain (stochastic).
- It is an extension of Markov chains by adding actions and rewards.
- MDP helps an agent to learn the best policy (sequence of actions) that maximizes the long-term expected reward.
- It is widely used in reinforcement learning to model the interaction between an agent and an environment.

2. Need of MDP

- Many real-world problems are sequential and involve uncertainty in outcomes.
- Need a model to decide the best action at each step, considering both current and future rewards.
- Helps in finding an optimal policy that maximizes long-term cumulative reward instead of only immediate reward.
- Useful in dynamic and uncertain environments such as robotics, games, navigation, resource allocation etc.

3. Assumptions / Characteristics

1. Markov Property: Future state depends only on the current state and action, not on the past history.
2. Discrete (or continuous) time steps.
3. Environment is stochastic (transition is probabilistic).
4. Set of states, actions, rewards and transition probabilities are well defined.
5. Agent aims to maximize expected cumulative reward (usually discounted).

4. Components of MDP

An MDP is defined by a 5-tuple (S, A, P, R, γ)

Component	Description
S : Set of states	Finite or infinite set of all possible situations in the environment.
A : Set of actions	Set of actions available to the agent in each state.
P : Transition probability function	$P(s' \mid s, a)$ = Probability of moving to next state s' from state s after taking action a.
R : Reward function	$R(s, a)$ = Immediate reward received after taking action a in state s.
γ : Discount factor	$0 \le \gamma < 1$, represents the importance of future rewards. A value close to 1 gives more importance to long-term rewards.

5. Model of MDP



Working (Interaction Cycle):

1. Agent observes current state s_t.
2. Chooses an action a_t (according to its policy π).
3. Environment moves to next state s_{t+1} with probability $P(s' \mid s, a)$.
4. Agent receives reward r_{t+1}.
5. Goal: find an optimal policy $\pi^*(s)$ that maximizes expected cumulative reward.

6. Mathematical Formulation

- Objective: Find a policy π that maximizes the expected discounted cumulative reward.
- Value function:

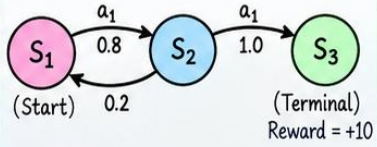
$$V^{\pi}(s) = E_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s\right]$$

- Bellman equation (for optimal value function):

$$V^*(s) = \max_{a \in A}\left[R(s, a) + \gamma \sum_{s' \in S} P(s' \mid s, a)\, V^*(s')\right]$$

- $V^*(s)$: maximum expected cumulative reward from state s.
- $\pi^*(s)$: optimal policy that gives the best action in each state.

7. Example (Simple MDP)



8. Applications of MDP

1. Robotics

Path planning, motion control, robot navigation

2. Game Playing

Strategic decision making in games (Chess, Go, etc.)

3. Autonomous Vehicles

Navigation, route planning, obstacle avoidance

4. Resource Management

Allocation of resources (cloud computing, energy management)

5. Healthcare

Treatment planning, patient management under uncertainty

6. Finance

Portfolio management, trading strategies, risk management

Q5. Bagging and Boosting / ensemble methods / bias–variance trade-off

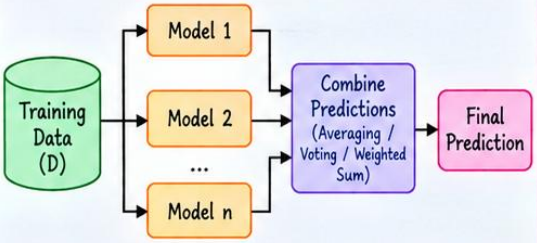
ANS

Ensemble Methods : Bagging and Boosting / Bias-Variance Trade-off

1. Introduction

- Ensemble learning combines multiple learning models (base learners) to get a better prediction than a single model.
- It improves accuracy, stability and generalization performance.
- Main ensemble methods:
  - Bagging (Bootstrap Aggregating)
  - Boosting
- These methods help to reduce the errors of individual models by combining their predictions.

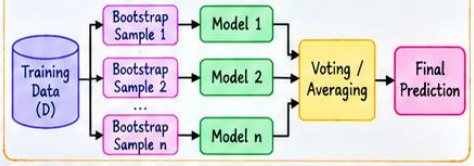
2. What is Ensemble Learning ?



- Multiple models are trained and their outputs are combined.
- For classification – majority voting is used.
- For regression – averaging is used.

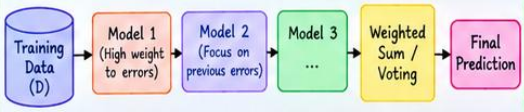
3. Bagging (Bootstrap Aggregating)

- Uses multiple training datasets created by bootstrapping (sampling with replacement).
- Trains each base learner independently and in parallel.
- Final prediction is obtained by averaging (for regression) or majority voting (for classification).
- Main algorithm: Random Forest.
- Reduces variance and helps to prevent overfitting.



4. Boosting

- Trains weak learners sequentially, where each new model focuses on the errors made by the previous models.
- Assigns higher weights to misclassified instances.
- Final prediction is a weighted sum (or weighted voting) of all weak learners.
- Main algorithms: AdaBoost, Gradient Boosting, XGBoost.
- Reduces bias and usually improves accuracy.

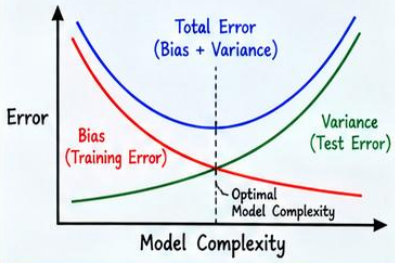


5. Comparison : Bagging vs Boosting

Feature	Bagging	Boosting
Training approach	Parallel	Sequential
Data sampling	Bootstrap sampling	Re-weights data based on errors
Base learners	Independent models	Dependent models
Combining method	Averaging / Majority voting	Weighted voting
Main algorithm	Random Forest	AdaBoost, Gradient Boosting, XGBoost
Mainly reduces	Variance	Bias
Sensitivity to noise/outliers	Less sensitive	More sensitive
Overfitting	Less likely	Can overfit if not tuned properly

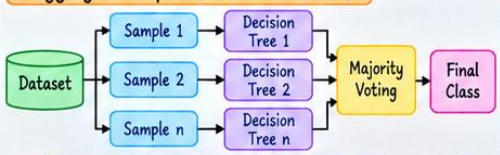
6. Bias – Variance Trade-off

- Bias is the error due to incorrect assumptions in the model (underfitting).
- Variance is the error due to high sensitivity to training data (overfitting).
- Ensemble methods help to achieve a balance between bias and variance.



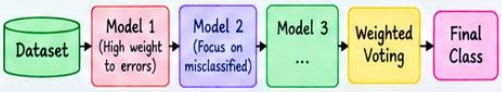
7. Illustration Example

Bagging Example (Random Forest)



- Each tree is trained on a different bootstrap sample and final class is decided by majority voting.

Boosting Example (AdaBoost)



- Each model gives more weight to misclassified instances of the previous model.
- Final prediction is a weighted vote of all models.

8. Conclusion

- Ensemble methods like bagging and boosting combine multiple models to improve prediction performance.
- Bagging reduces variance and is less prone to overfitting.
- Boosting reduces bias and focuses on difficult examples.
- Together, they help to achieve a better bias-variance trade-off and provide more accurate and robust models.

Q6. Defuzzification methods (including centroid method

ANS

Defuzzification Methods (including Centroid Method)

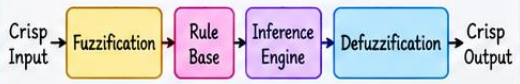
1. Introduction

- Defuzzification is the process of converting a fuzzy set (fuzzy output) into a single crisp value used for real world decision making.
- In fuzzy inference system, after applying fuzzy rules, we get an aggregated fuzzy output (membership function).
- Defuzzification converts this fuzzy output into a precise numerical value (crisp output).

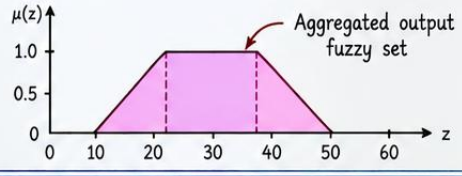
2. Need for Defuzzification

- Real world systems require a single numerical (crisp) output for control actions (e.g., speed, temperature).
- Fuzzy inference produces a fuzzy set as output, which cannot be directly used in real systems.
- Defuzzification provides a meaningful crisp value representing the fuzzy result.
- It acts as an interface between fuzzy world and real world.

3. Position in Fuzzy Inference System



4. Aggregated Fuzzy Output (Example)



5. Defuzzification Methods

(1) Centroid Method (Center of Area)

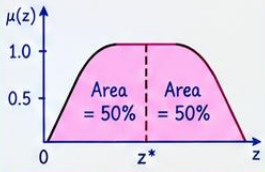
- It gives the center of gravity of the area under the membership function.
- Most popular and widely used method.
- It considers all the values of the fuzzy set.

Formula:

$$z^* = \frac{\int z\,\mu(z)\,dz}{\int \mu(z)\,dz}$$

(2) Bisector of Area (BOA)

- It divides the total area under the membership function into two equal halves.



z* such that

$$\int_a^{z^*} \mu(z)\,dz = \int_{z^*}^{b} \mu(z)\,dz$$

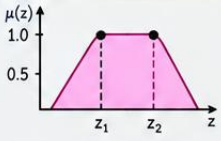
(3) Mean of Maximum (MOM)

- It takes the average of all the values of z for which the membership function is maximum.

Formula:

$$z^* = \frac{1}{n}\sum_{i=1}^{n} z_i$$

where z_i are the values at which $\mu(z)$ is maximum.

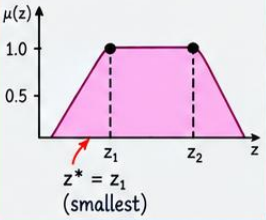


$z^* = (z_1 + z_2)/2$

(4) Smallest of Maximum (SOM)

- It selects the smallest value of z for which the membership function is maximum.

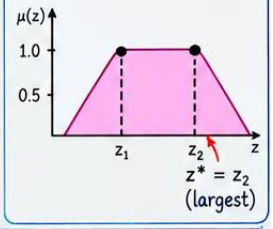
$$z^* = \min\{z \mid \mu(z) = \mu_{max}\}$$



(5) Largest of Maximum (LOM)

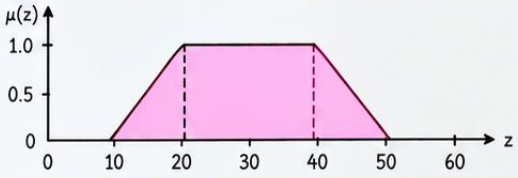
- It selects the largest value of z for which the membership function is maximum.

$$z^* = \max\{z \mid \mu(z) = \mu_{max}\}$$



6. Numerical Example (Centroid Method)

Consider an aggregated output fuzzy set (trapezoidal):



Using centroid formula:

$$z^* = \frac{\int z\,\mu(z)\,dz}{\int \mu(z)\,dz}$$

For this symmetric trapezoid, centroid is at the center:

$z^* = (20 + 40)/2 = 30$

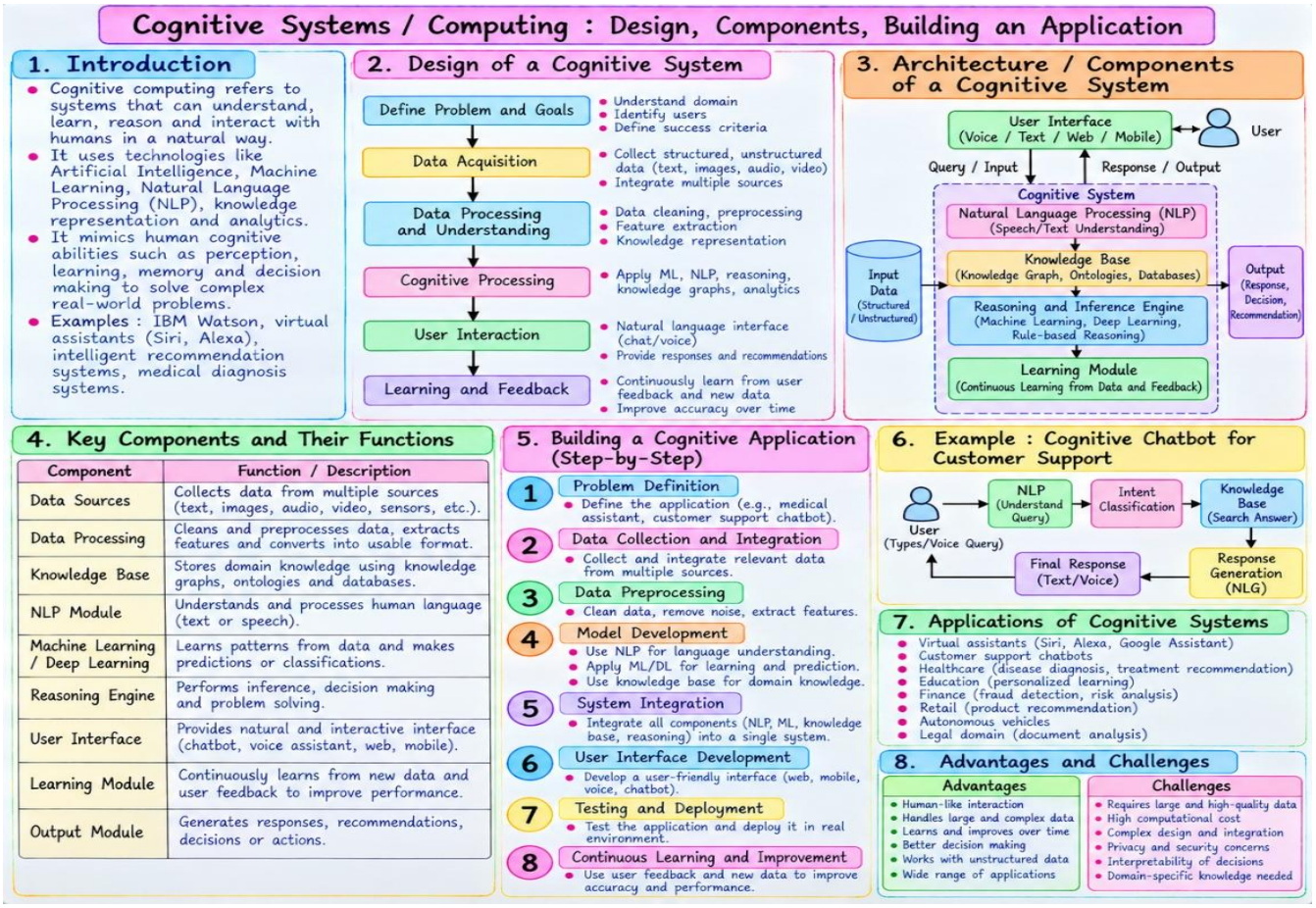
Crisp output = 30

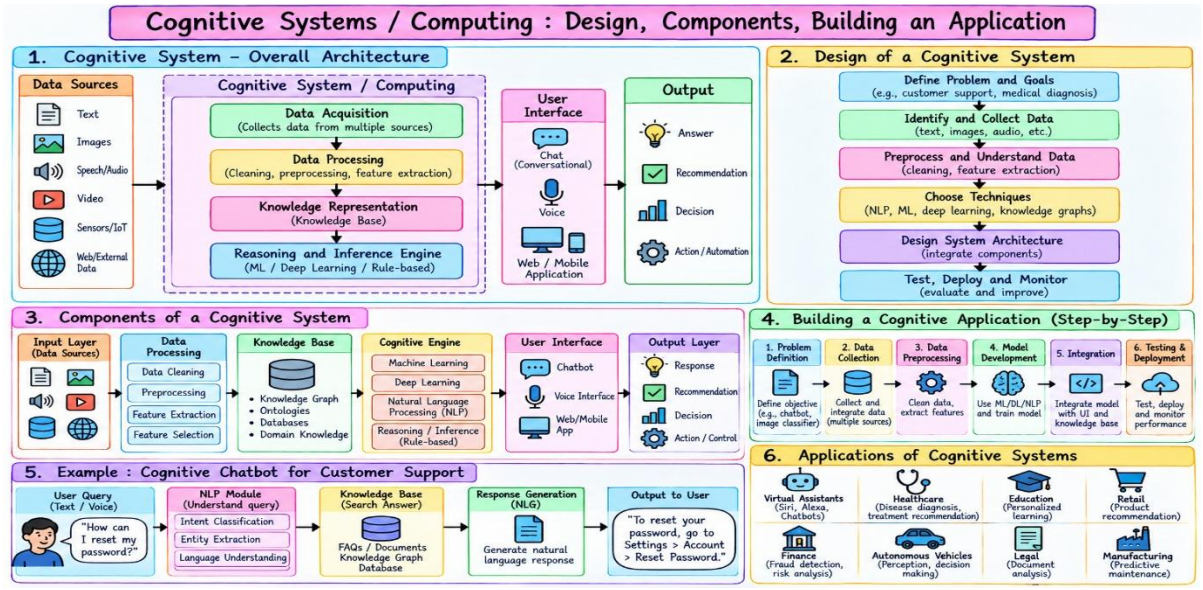
7. Applications

- Control systems (e.g., temperature control, motor speed control)
- Industrial automation
- Decision making systems
- Consumer electronics (washing machines, AC)
- Medical diagnosis systems
- Image processing
- Robotics and autonomous vehicles
- Expert systems

Q7. Cognitive systems/computing: design, components, building an application.

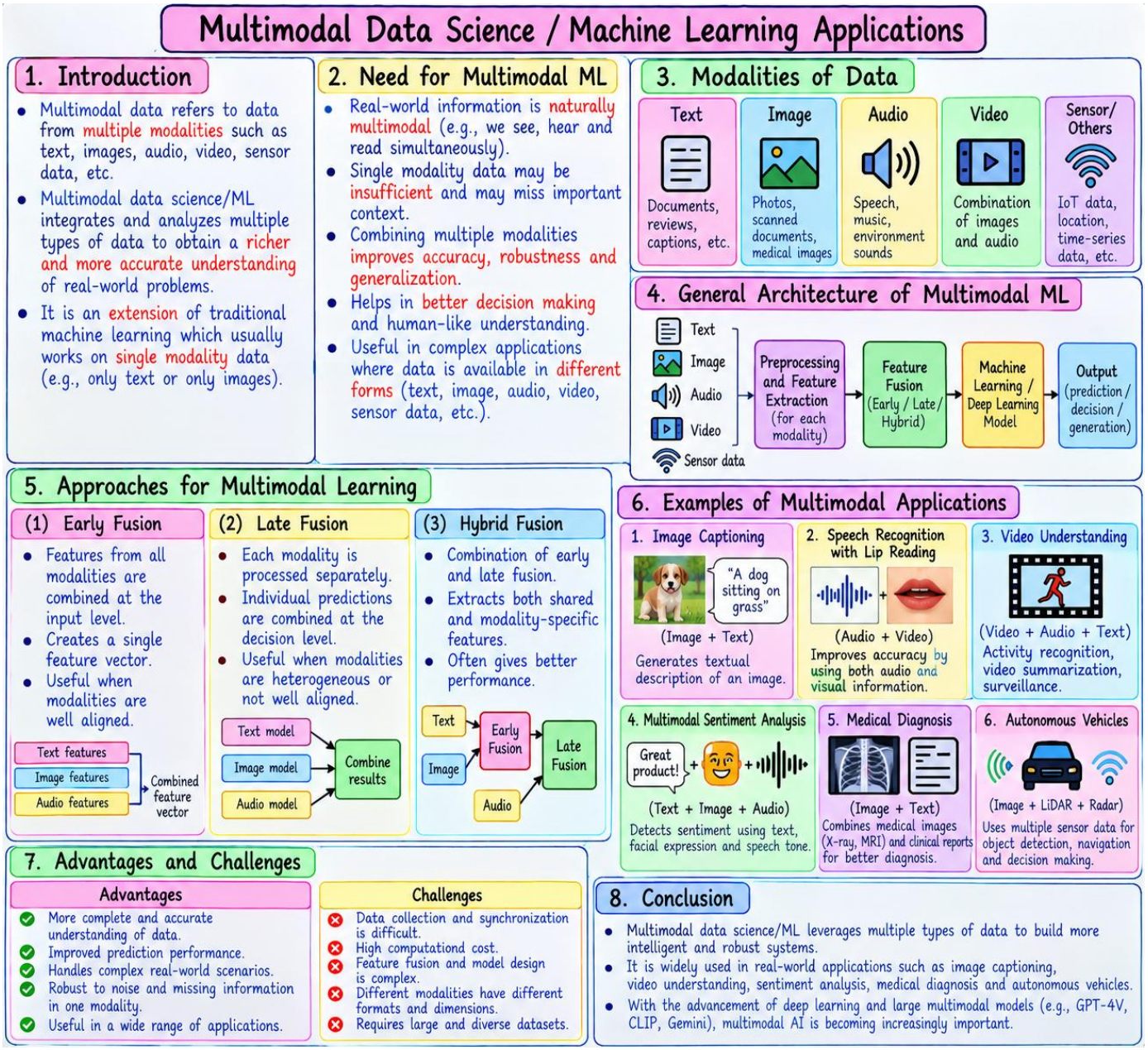
ANS





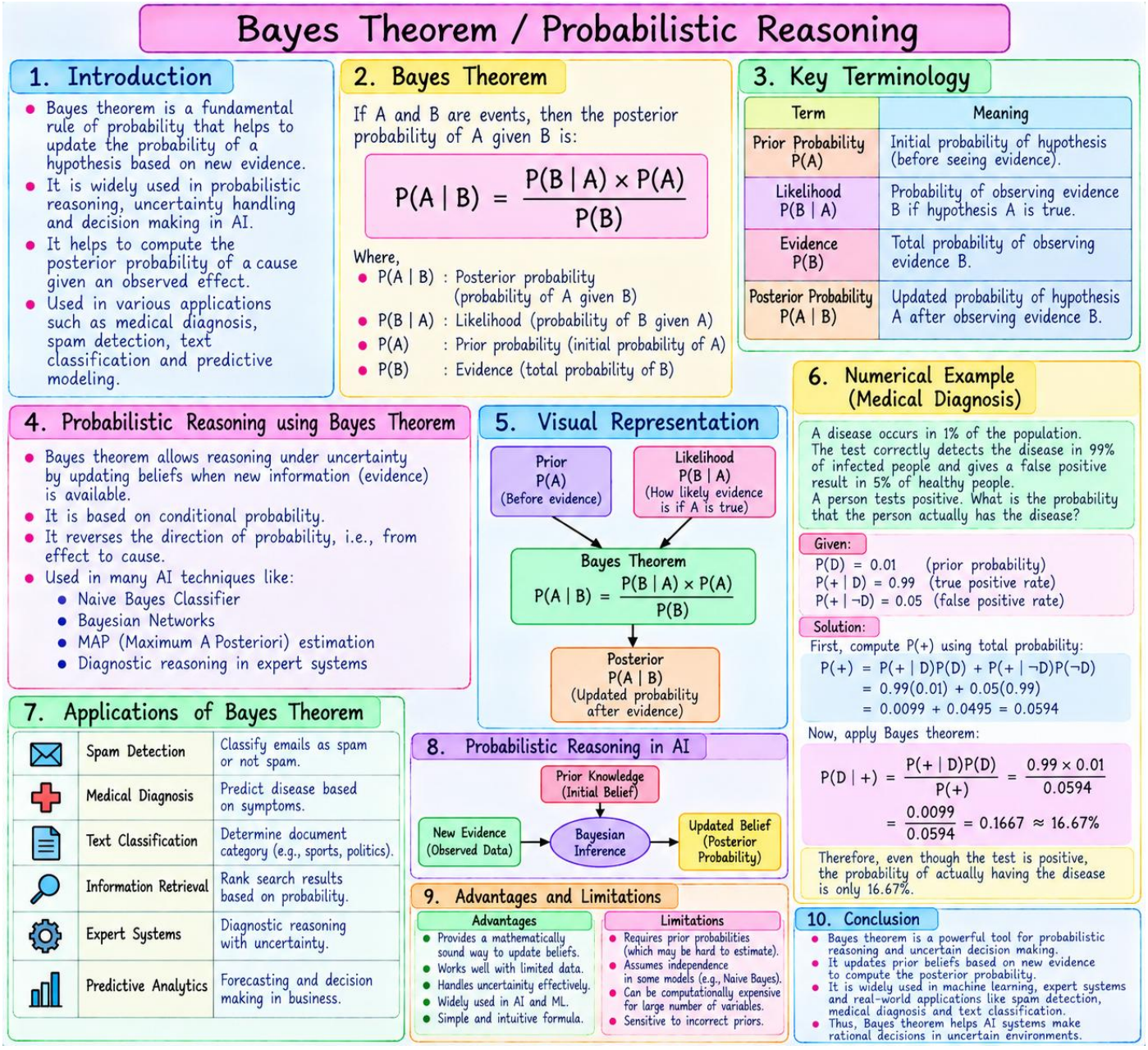
Q8. Multimodal data science / machine-learning applications

ANS



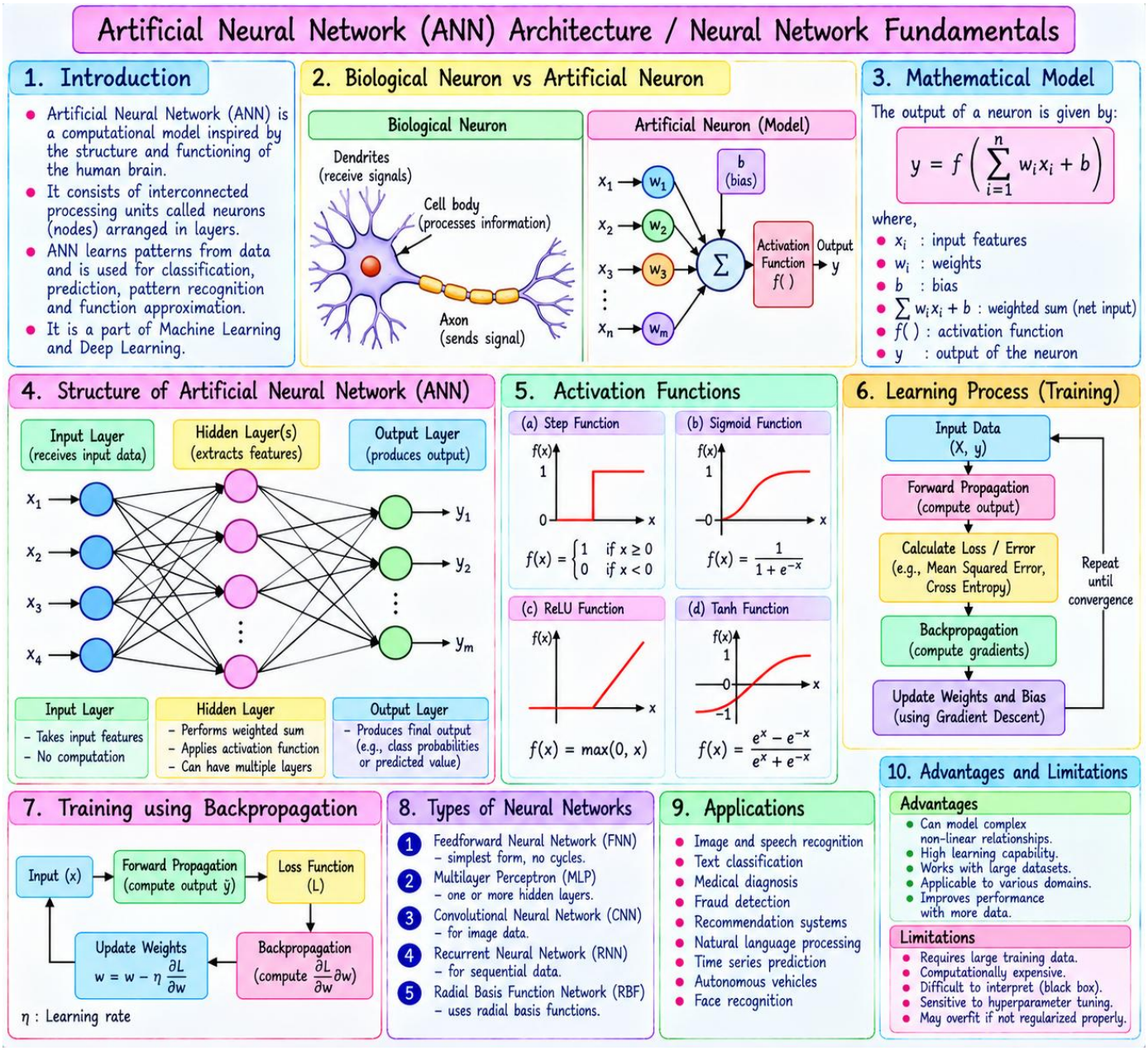
Q9 Bayes theorem / probabilistic reasoning

ANS



Q10. ANN architecture / neural-network fundamentals

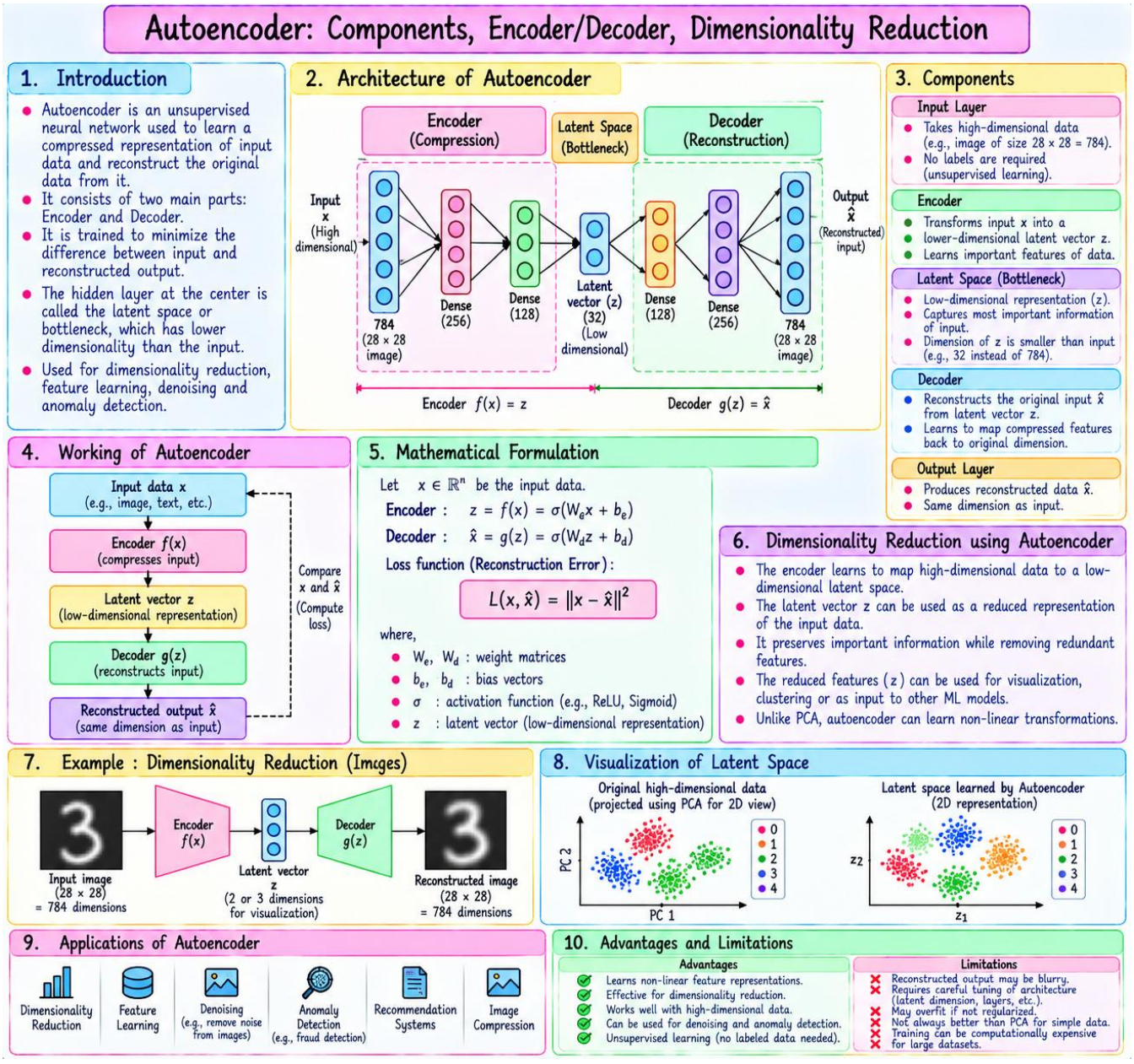
ANS

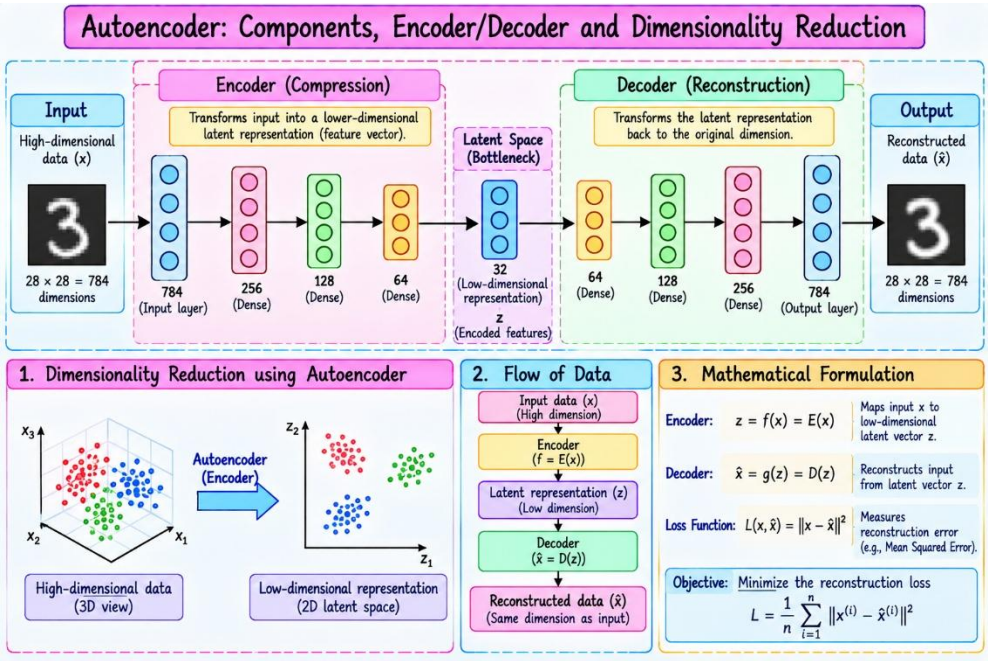


2 TIME

Q11. Autoencoder: components, encoder/decoder, dimensionality reduction

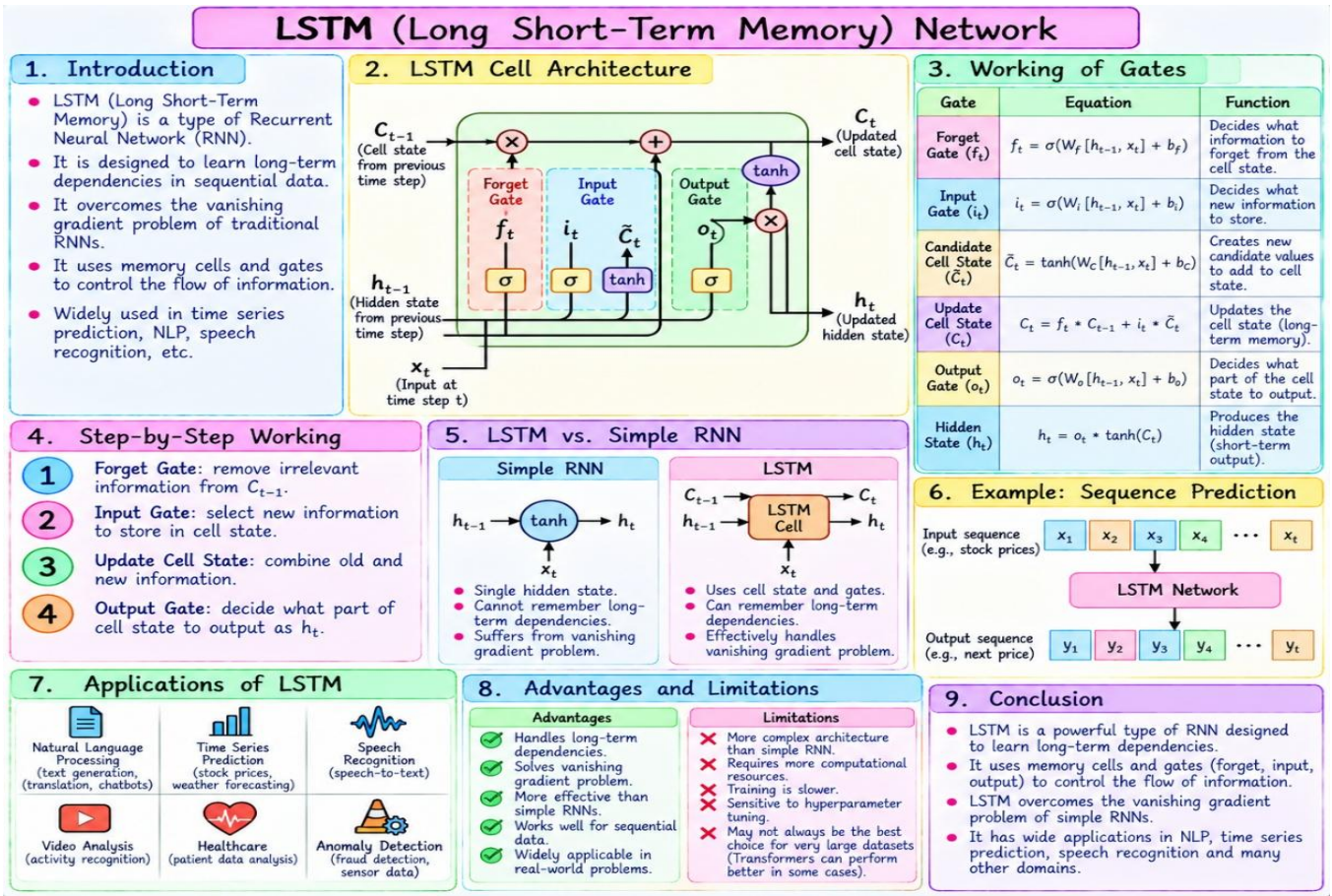
ANS





Q12. LSTM / Long Short-Term Memory network

ANS



Q13. McCulloch–Pitts neuron / AND or XOR logic

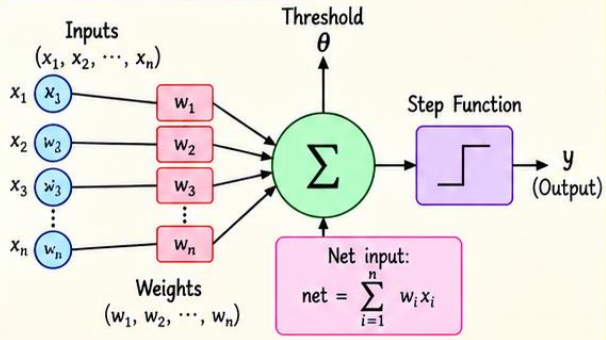
ANS

Mc Culloch – Pitts (M-P) Neuron / AND or XOR Logic

1. Introduction

- McCulloch-Pitts (M-P) neuron was proposed by Warren McCulloch and Walter Pitts in 1943.
- It is a simple mathematical model of a biological neuron.
- It takes binary inputs (0 or 1), multiplies them with weights and produces a binary output using a threshold function.
- It is the first model of an artificial neural network.
- Used for understanding how neural networks can implement logical functions.

2. M-P Neuron Model



3. Mathematical Model

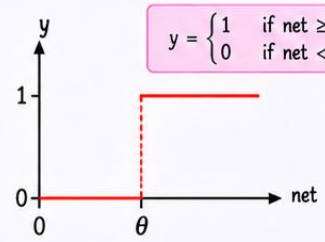
The output of M-P neuron is given by:

$$y = \begin{cases} 1 & \text{if } \sum_{i=1}^{n} w_i x_i \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

where,

- x_i : binary input (0 or 1)
- w_i : weight associated with input x_i
- θ : threshold value
- $\sum w_i x_i$: net input (weighted sum)
- y : binary output (0 or 1)

4. Step Function

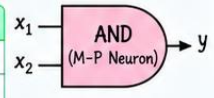


5. AND Logic using M-P Neuron

AND gate gives output 1 only when all inputs are 1.
Choose weights and threshold:

$w_1 = 1, \quad w_2 = 1, \quad \theta = 2$

x_1	x_2	net = $x_1 + x_2$	Output y
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2	1



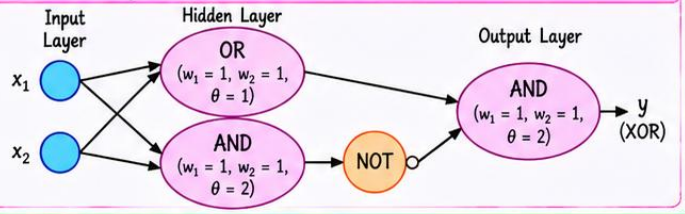
$y = 1$ if $x_1 + x_2 \geq 2$
else $y = 0$

6. XOR Logic using M-P Neuron

- A single M-P neuron cannot implement XOR function because XOR is not linearly separable.
- XOR can be implemented using a multi-layer M-P network (two-layer network).
- Idea: XOR = (x_1 OR x_2) AND (NOT (x_1 AND x_2))

x_1	x_2	XOR Output y
0	0	0
0	1	1
1	0	1
1	1	0

7. Two-Layer M-P Network for XOR



8. Verification of XOR Network

x_1	x_2	OR ($x_1 + x_2 \geq 1$)	AND ($x_1 + x_2 \geq 2$)	NOT (AND)	NOT (AND)	Final AND (OR · NOT)	Output y (XOR)
0	0	0	0	1	0	0	0
0	1	1	0	1	1	1	1
1	0	1	0	1	1	1	1
1	1	1	1	0	0	0	0

9. Advantages and Limitations

Advantages

- ✓ Simple and easy to understand.
- ✓ First mathematical model of a neuron.
- ✓ Can implement basic logic gates (AND, OR, NOT).
- ✓ Foundation for modern neural networks.

Limitations

- ✗ Only binary inputs and outputs.
- ✗ Cannot solve non-linearly separable problems (e.g., XOR with single neuron).
- ✗ No learning rule (weights must be set manually).
- ✗ Very limited computational capability compared to modern neural networks.

10. Conclusion

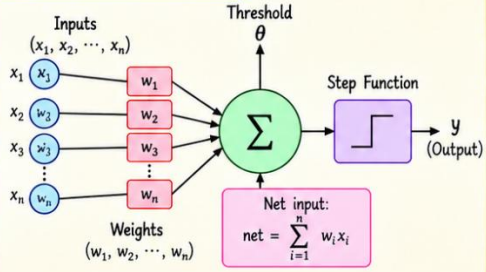
- M-P neuron is a simple threshold-based model of a biological neuron.
- It can implement logic gates like AND, OR and NOT.
- XOR function cannot be implemented by a single M-P neuron, but it can be implemented using a multi-layer network.
- Although it has limitations, it laid the foundation for modern artificial neural networks and deep learning.

Mc Culloch – Pitts (M-P) Neuron / AND or XOR Logic

1. Introduction

- McCulloch-Pitts (M-P) neuron was proposed by Warren McCulloch and Walter Pitts in 1943.
- It is a simple mathematical model of a biological neuron.
- It takes binary inputs (0 or 1), multiplies them with weights and produces a binary output using a threshold function.
- It is the first model of an artificial neural network.
- Used for understanding how neural networks can implement logical functions.

2. M-P Neuron Model



3. Mathematical Model

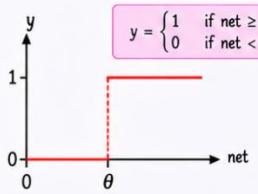
The output of M-P neuron is given by:

$$y = \begin{cases} 1 & \text{if } \sum_{i=1}^{n} w_i x_i \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

where,

- x_i : binary input (0 or 1)
- w_i : weight associated with input x_i
- θ : threshold value
- $\sum w_i x_i$: net input (weighted sum)
- y : binary output (0 or 1)

4. Step Function

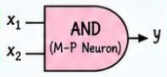


5. AND Logic using M-P Neuron

AND gate gives output 1 only when all inputs are 1.
Choose weights and threshold:

$w_1 = 1, \quad w_2 = 1, \quad \theta = 2$

x_1	x_2	$net = x_1 + x_2$	Output y
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2	1



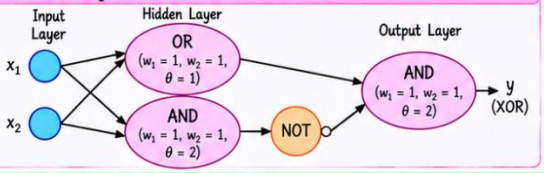
$y = 1$ if $x_1 + x_2 \geq 2$
else $y = 0$

6. XOR Logic using M-P Neuron

- A single M-P neuron cannot implement XOR function because XOR is not linearly separable.
- XOR can be implemented using a multi-layer M-P network (two-layer network).
- Idea: XOR = (x_1 OR x_2) AND (NOT (x_1 AND x_2))

x_1	x_2	XOR Output y
0	0	0
0	1	1
1	0	1
1	1	0

7. Two-Layer M-P Network for XOR



8. Verification of XOR Network

x_1	x_2	OR $(x_1 + x_2 \geq 1)$	AND $(x_1 + x_2 \geq 2)$	NOT (AND)	z_1 NOT (AND)	Final AND (OR · NOT)	Output y (XOR)
0	0	0	0	1	0	0	0
0	1	1	0	1	1	1	1
1	0	1	0	1	1	1	1
1	1	1	1	0	0	0	0

9. Advantages and Limitations

Advantages

- Simple and easy to understand.
- First mathematical model of a neuron.
- Can implement basic logic gates (AND, OR, NOT).
- Foundation for modern neural networks.

Limitations

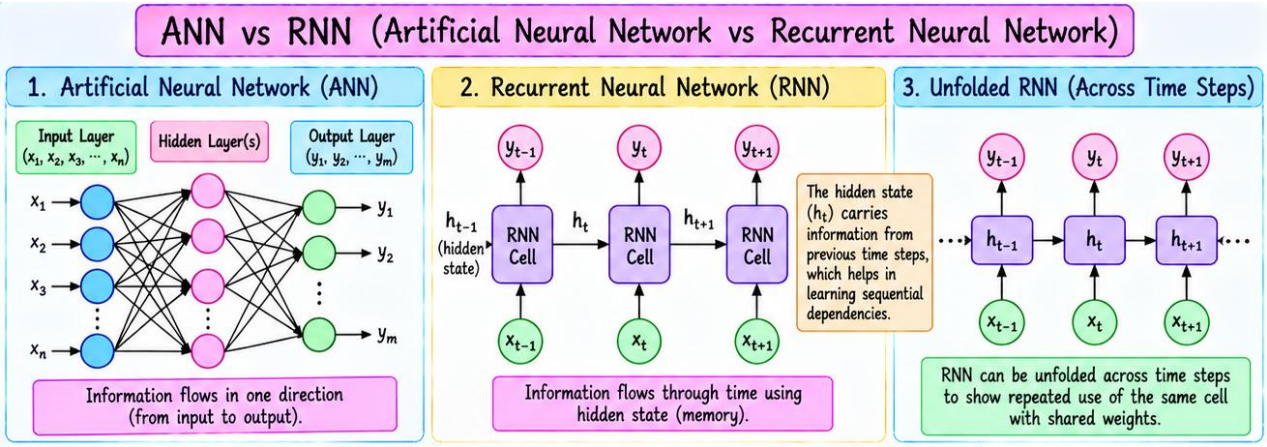
- Only binary inputs and outputs.
- Cannot solve non-linearly separable problems (e.g., XOR with single neuron).
- No learning rule (weights must be set manually).
- Very limited computational capability compared to modern neural networks.

10. Conclusion

- M-P neuron is a simple threshold-based model of a biological neuron.
- It can implement logic gates like AND, OR and NOT.
- XOR function cannot be implemented by a single M-P neuron, but it can be implemented using a multi-layer network.
- Although it has limitations, it laid the foundation for modern artificial neural networks and deep learning.

Q14. ANN vs RNN

ANS

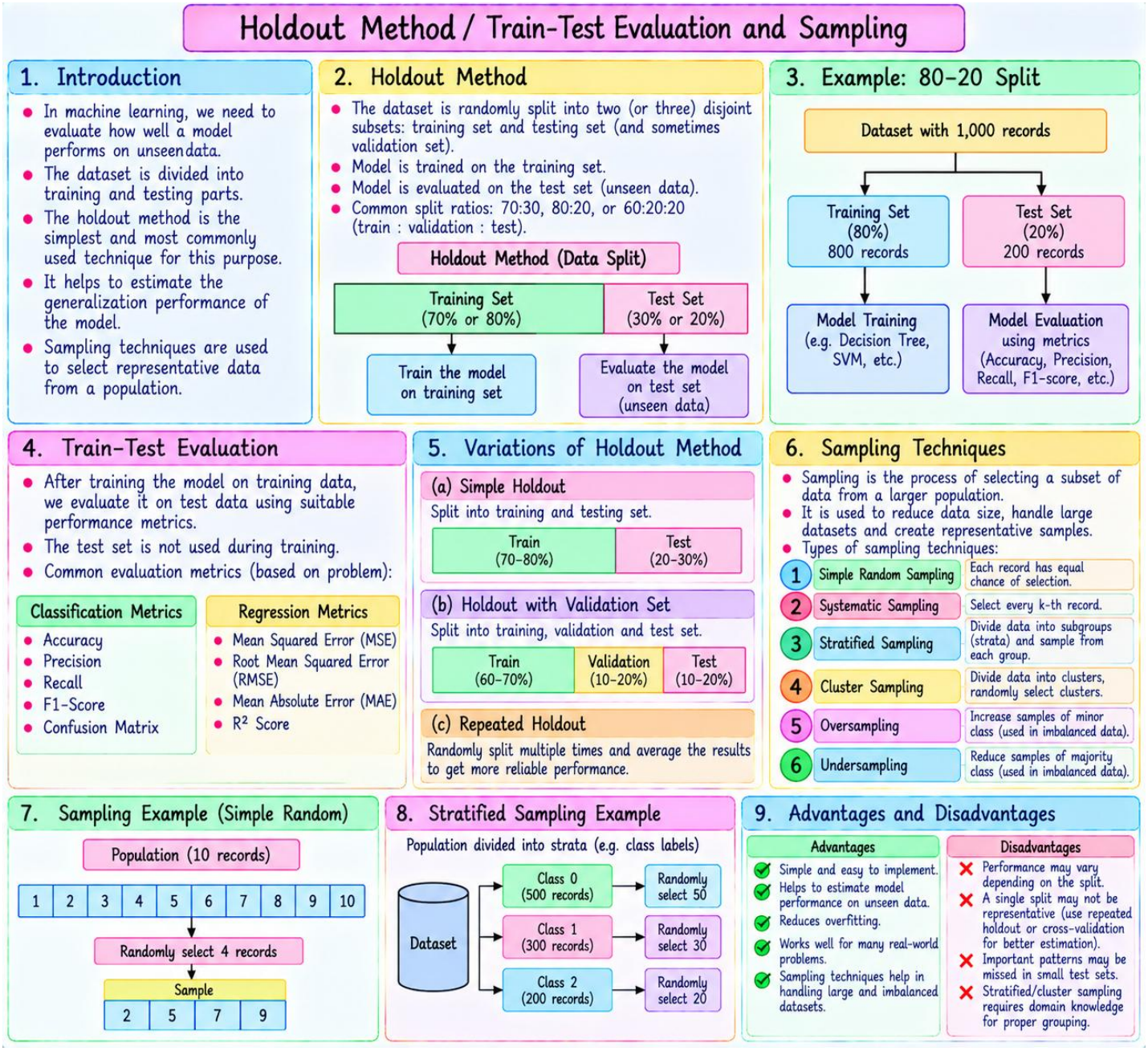


4. Comparison between ANN and RNN (15 Points)

Sr. No.	Aspect	ANN (Artificial Neural Network)	RNN (Recurrent Neural Network)
1.	Full Form	Artificial Neural Network	Recurrent Neural Network
2.	Architecture	Feedforward network (input → hidden → output).	Has feedback connections (recurrent links) to maintain hidden state (memory).
3.	Data Type	Works on fixed-size, independent input data.	Designed for sequential or time-series data.
4.	Information Flow	Unidirectional (from input to output).	Information flows across time steps using hidden state.
5.	Memory	No memory of previous inputs.	Maintains memory (hidden state) of previous inputs.
6.	Weights	Different weights for different layers.	Same weights are shared across all time steps.
7.	Input Length	Fixed-size input vector.	Can handle variable-length sequences.
8.	Use Cases	Image classification, pattern recognition, regression, classification (non-sequential data).	Time series prediction, speech recognition, language modeling, text generation, video analysis (sequential data).
9.	Example Models	MLP (Multilayer Perceptron), CNN (also a type of ANN).	Simple RNN, LSTM, GRU.
10.	Training Method	Standard backpropagation.	Backpropagation Through Time (BPTT).
11.	Ability to Handle Sequential Dependencies	Cannot capture long-term dependencies (no memory).	Can capture long-term dependencies (especially LSTM/GRU).
12.	Computational Complexity	Generally lower (for fixed-size input).	Higher due to recurrent connections and sequence processing.
13.	Common Problems	May underfit or overfit like any neural network.	Vanishing gradient and exploding gradient (solved by LSTM/GRU).
14.	Execution	Processes all inputs at once (parallelizable).	Processes inputs step-by-step (sequential computation).
15.	Applications in Real World	Image recognition, medical diagnosis, fraud detection, recommendation systems (non-sequential).	Speech recognition, machine translation, stock price prediction, text generation, video captioning (sequential).

Q15. Holdout method / train-test evaluation and sampling

ANS



Q16. Class imbalance and ways to handle it.

ANS

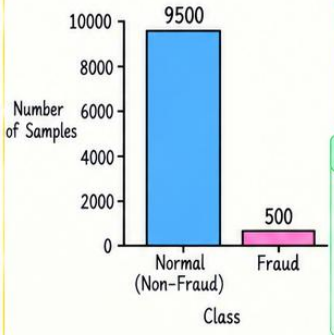
Class Imbalance and Ways to Handle It

1. Introduction

- In many real-world classification problems, the number of samples in different classes is not equal.
- This is called class imbalance.
- The majority class has many samples, while the minority class has very few samples.
- It can lead to biased models that perform well on the majority class but poorly on the minority class.
- Common in domains like fraud detection, medical diagnosis, rare event prediction, etc.

2. Example of Class Imbalance

Example: Fraud Detection



Here, the dataset is highly imbalanced:
Normal class = 95%
Fraud class = 5%

Imbalance Ratio

$$IR = \frac{\text{Majority class samples}}{\text{Minority class samples}}$$

$$IR = \frac{9500}{500} = 19 : 1$$

3. Effect of Class Imbalance

- Model tends to predict the majority class for most of the samples.
- High accuracy can be misleading.
- Minority class (important class) is poorly detected.
- Leads to low recall, low F1-score for the minority class.

Example: Confusion Matrix (Imbalanced Data)

Actual Class	Predicted Class	
	Normal	Fraud
Normal	9400 (TN)	100 (FP)
Fraud	400 (FN)	100 (TP)

4. Evaluation Metrics for Imbalanced Data

- Accuracy is not sufficient (misleading).
- Use metrics that consider minority class performance.
- Important metrics:

1. **Precision** – proportion of correct positive predictions.
2. **Recall (Sensitivity)** – proportion of actual positives detected.
3. **F1-Score** – harmonic mean of precision and recall.
4. **Confusion Matrix**
5. **ROC-AUC and PR-AUC** (Precision-Recall Curve) – more informative for imbalanced data.

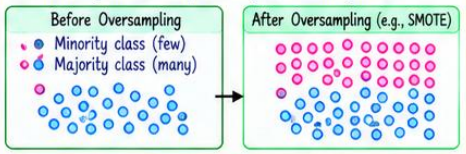
5. Ways to Handle Class Imbalance

1. Data Level Methods (Resampling)
2. Algorithm Level Methods (Cost-Sensitive Learning)
3. Ensemble Methods
4. Evaluation and Other Techniques

6. Data Level Methods (Resampling)

(a) Oversampling (increase minority class)

- Random Oversampling – duplicate minority samples.
- SMOTE (Synthetic Minority Over-sampling Technique) – create synthetic samples using nearest neighbors.



(b) Undersampling (reduce majority class)

- Random Undersampling – remove some majority class samples.
- Tomek Links, NearMiss – intelligent undersampling techniques.

7. Algorithm Level Methods

- Assign higher misclassification cost to minority class.
- Use class weights in algorithms (e.g., SVM, Decision Tree, Logistic Regression).

Cost-Sensitive Learning

Modify the loss function by giving higher weight to minority class samples.

Example: In scikit-learn, use class_weight='balanced'.

8. Ensemble Methods

- Combine multiple models to improve prediction of minority class.
- Examples:

1. **Balanced Random Forest** – uses balancing in each bootstrap sample.
2. **EasyEnsemble** – builds multiple models on different balanced subsets.
3. **AdaBoost with class weights** – focuses more on misclassified minority samples.

9. Other Techniques

- Use anomaly detection for very rare classes.
- Generate synthetic data (GANs).
- Adjust decision threshold for classification.
- Use specialized algorithms (e.g., One-Class SVM).
- Combine resampling + cost-sensitive learning for better results.

10. Advantages and Disadvantages

Advantages

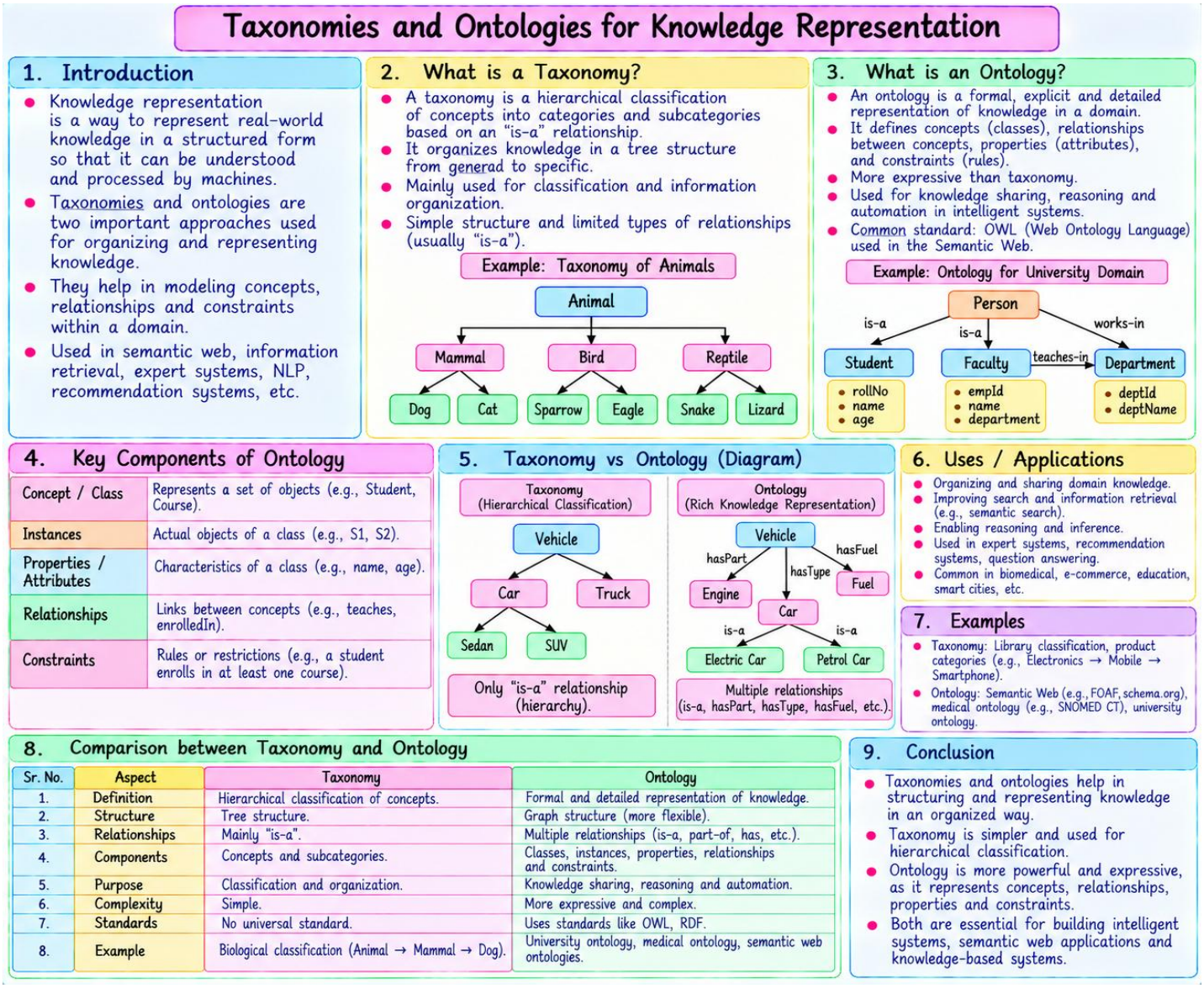
- ✓ Improves detection of minority class.
- ✓ Better model performance on important/rare events.
- ✓ Helps in real-world applications (e.g., fraud, medical diagnosis).

Disadvantages

- ✗ Oversampling may lead to overfitting.
- ✗ Undersampling may lose important data.
- ✗ Cost-sensitive methods require tuning.
- ✗ No single method works best for all problems.

Q17. Taxonomies and ontologies for knowledge representation.

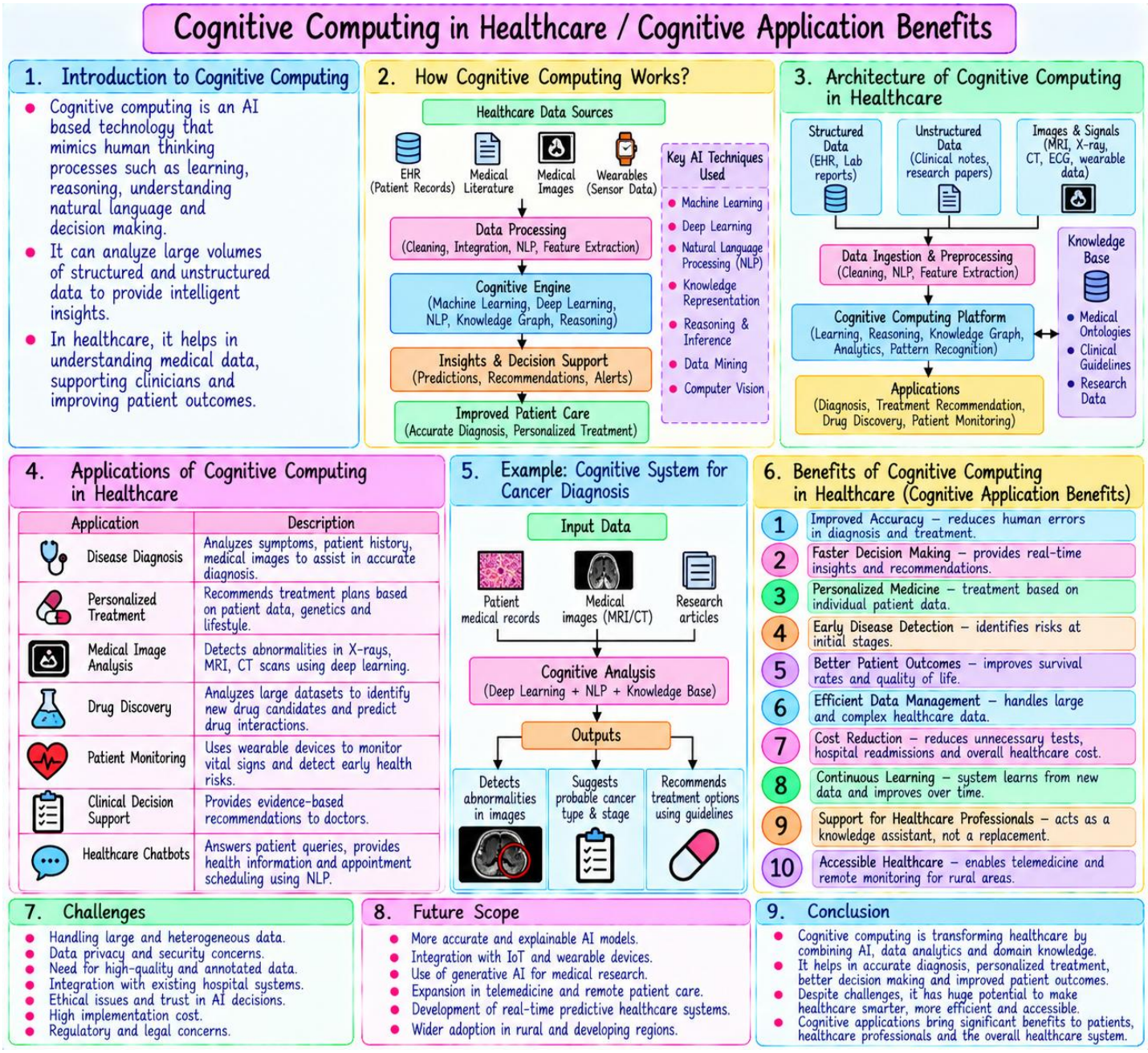
ANS



- Taxonomy: Organizes concepts in a hierarchical tree using is-a relationships.
- Ontology: Represents concepts, instances, properties, relationships, and constraints in a knowledge domain.
- Main difference: Taxonomy mainly classifies information, whereas ontology represents richer relationships and supports reasoning.
- Applications: Semantic Web, information retrieval, expert systems, healthcare, recommendation systems, and knowledge-based AI.

Q18. Cognitive computing in healthcare / cognitive application benefits.

ANS



UT 2

Q1. Differentiate between Fuzzyness and Probability

ANS

Difference Between Fuzziness and Probability (10 Points)

No.	Basis of Difference	Fuzziness (Fuzzy Logic)	Probability	Example / Diagram
1	Meaning	Deals with vagueness, imprecision and ambiguity in information.	Deals with randomness and uncertain events.	**Fuzziness:** "Temperature is high" (vague term) **Probability:** "It will rain" (random event)
2	Nature of Uncertainty	Uncertainty due to vague or ambiguous data (e.g., linguistic terms).	Uncertainty due to random behaviour or incomplete information.	**Fuzziness:** "Speed is fast" (vague) **Probability:** "Coin may show head or tail" (random)
3	Value Representation	Represents degree of membership with values between 0 and 1 (0 = not a member, 1 = full member).	Represents likelihood of an event with values between 0 and 1 (0 = impossible, 1 = certain).	Fuzzy Membership: 0 — 0.5 — 1 (Not member … Full member) Probability: 0 — 0.5 — 1 (Impossible … Certain)
4	Interpretation	Indicates how much an element belongs to a set (partial truth).	Indicates the chance or frequency of occurrence of an event.	Membership value 0.7 → Temperature is high to degree 0.7 Probability 0.7 → 70% chance of rain
5	Theoretical Basis	Based on Fuzzy Set Theory (Zadeh, 1965).	Based on Probability Theory (Kolmogorov, 1933).	Linguistic terms: low, medium, high Mathematical model for random events.
6	Type of Data Handled	Handles linguistic, imprecise and continuous data.	Handles random, discrete or continuous data.	Examples: slow, fast, hot, cold (imprecise) Examples: dice roll, weather, machine failure (random)
7	Event / Set	An element can belong to multiple sets with different degrees (simultaneously).	An event either occurs or does not occur, with a certain probability (though in practice we use values between 0 and 1).	A temperature of 30°C can be "warm" (0.6) and "hot" (0.4) at the same time. A coin either shows head or tail, with probability 0.5 for each.
8	Rule / Model Used	Uses fuzzy rules, membership functions and linguistic variables.	Uses probability distributions and statistical models.	IF temperature is high THEN fan speed is fast (fuzzy rule) P(Rain) = 0.6 (using probability model)
9	Application Area	Used in control systems, expert systems, decision making and image processing.	Used in risk analysis, forecasting, medical diagnosis, machine learning and statistical modelling.	Air conditioner control using fuzzy logic. Weather prediction using probability.
10	Main Focus	Focuses on handling imprecise and uncertain human knowledge.	Focuses on modelling and analysing randomness and uncertainty.	How "high" the temperature is (vagueness). How likely it is that it will rain (randomness).

Difference Between Fuzziness and Probability

1. Fuzziness (Fuzzy Logic)

- Fuzziness deals with imprecise, vague or ambiguous information.
- It represents the degree to which an element belongs to a set.
- The value of membership can be anywhere between 0 and 1.
- It is used to handle uncertainty due to vagueness in human knowledge and linguistic terms (e.g., low, medium, high).
- It is based on Fuzzy Set Theory (Zadeh, 1965).

2. Probability

- Probability deals with randomness and uncertain events.
- It represents the likelihood of an event to occur.
- The probability value also lies between 0 and 1, but it indicates the chance or frequency of an event.
- It is used to handle uncertainty due to incomplete information or random behaviour.
- It is based on Probability Theory (Kolmogorov, 1933).

Key Idea

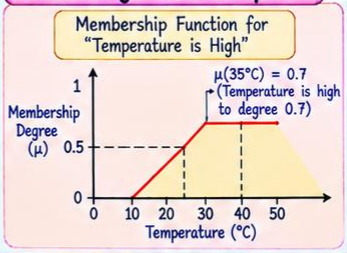
Fuzziness: How much an element belongs to a set (vagueness).

Probability: How likely an event will occur (randomness).

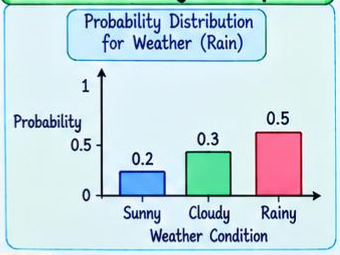
3. Difference Between Fuzziness and Probability

No.	Basis	Fuzziness	Probability
1.	Meaning	Deals with vagueness and imprecision.	Deals with randomness and likelihood of events.
2.	Nature of Uncertainty	Uncertainty due to vague or ambiguous data.	Uncertainty due to randomness or incomplete data
3.	Value Representation	Degree of membership (0 to 1).	Likelihood of occurrence (0 to 1).
4.	Interpretation	Indicates how much an element belongs to a set.	Indicates the chance or frequency of an event.
5.	Set Theory Used	Fuzzy Set Theory (Zadeh, 1965).	Classical Probability Theory (Kolmogorov, 1933).
6.	Example	Temperature is high (0.7), Age is young (0.5).	Probability of rain is 0.7, Probability of getting a head is 0.5.
7.	Type of Data	Linguistic, imprecise and continuous data.	Random, discrete or continuous data.
8.	Event / Set	An element can belong to multiple sets with different degrees.	An event either occurs or not, with a certain probability (though in practice we use probability values).
9.	Rule / Model	Uses fuzzy rules and membership functions.	Uses probability distributions and statistical models.
10.	Applications	Control systems, decision making, expert systems, image processing.	Risk analysis, forecasting, medical diagnosis, machine learning.

4. Fuzzy Set Example



5. Probability Example



6. Real Life Example Comparison

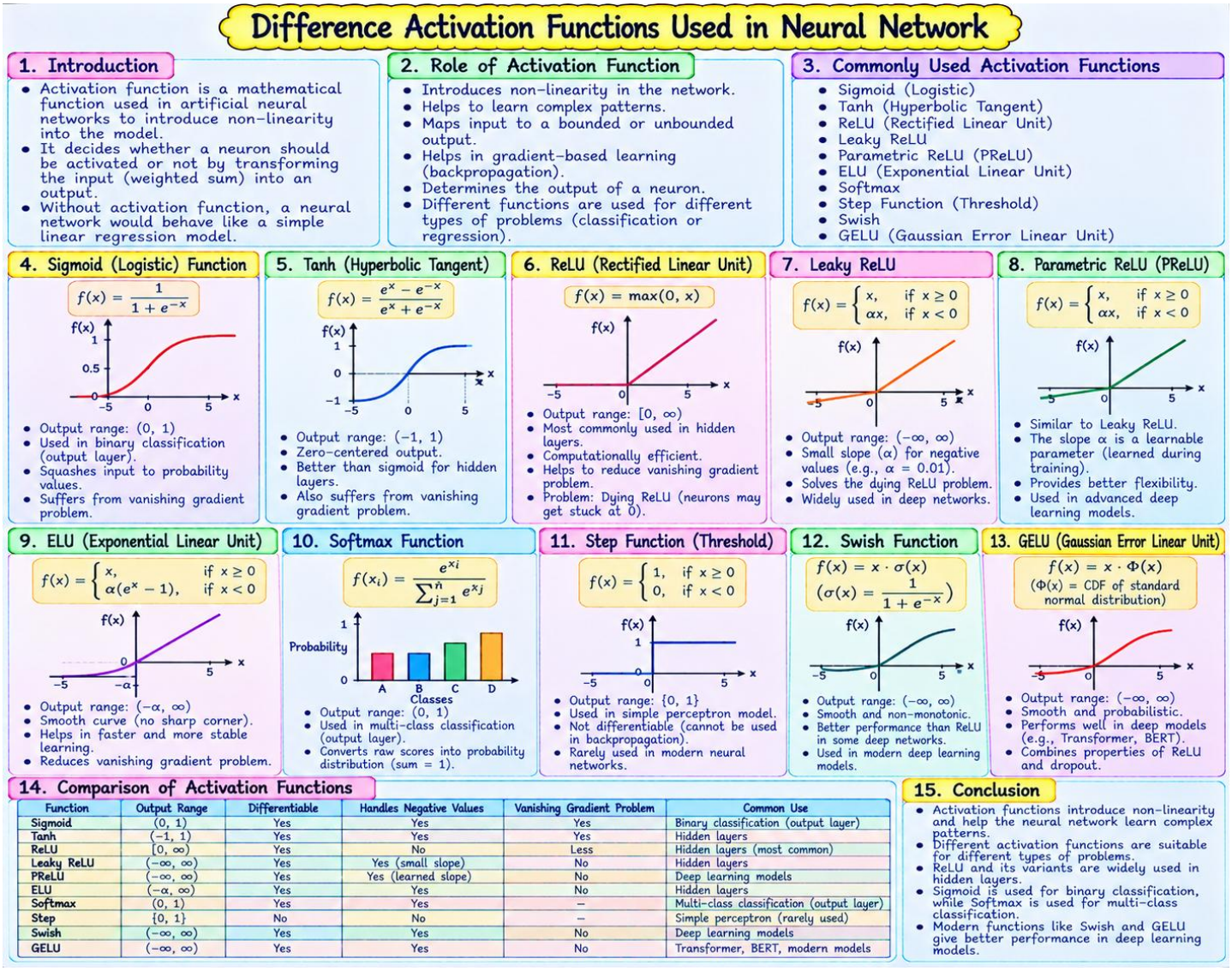
Situation	Fuzziness	Probability
Weather	Today is a warm day ($\mu = 0.6$)	Probability of rain is 0.6 (60% chance)
Age	A person is young ($\mu = 0.8$)	Probability that a person is 20 years old may be 0.1
Traffic	Traffic is heavy ($\mu = 0.7$)	Probability of a traffic jam is 0.7
Health	Risk is high ($\mu = 0.6$)	Probability of disease occurrence is 0.6

7. Conclusion

- Fuzziness handles vagueness and imprecision, while probability handles randomness and likelihood.
- Both concepts deal with uncertainty, but their nature and interpretation are different.
- Fuzzy logic is useful when information is linguistically imprecise, whereas probability theory is useful when events are random and based on chance.

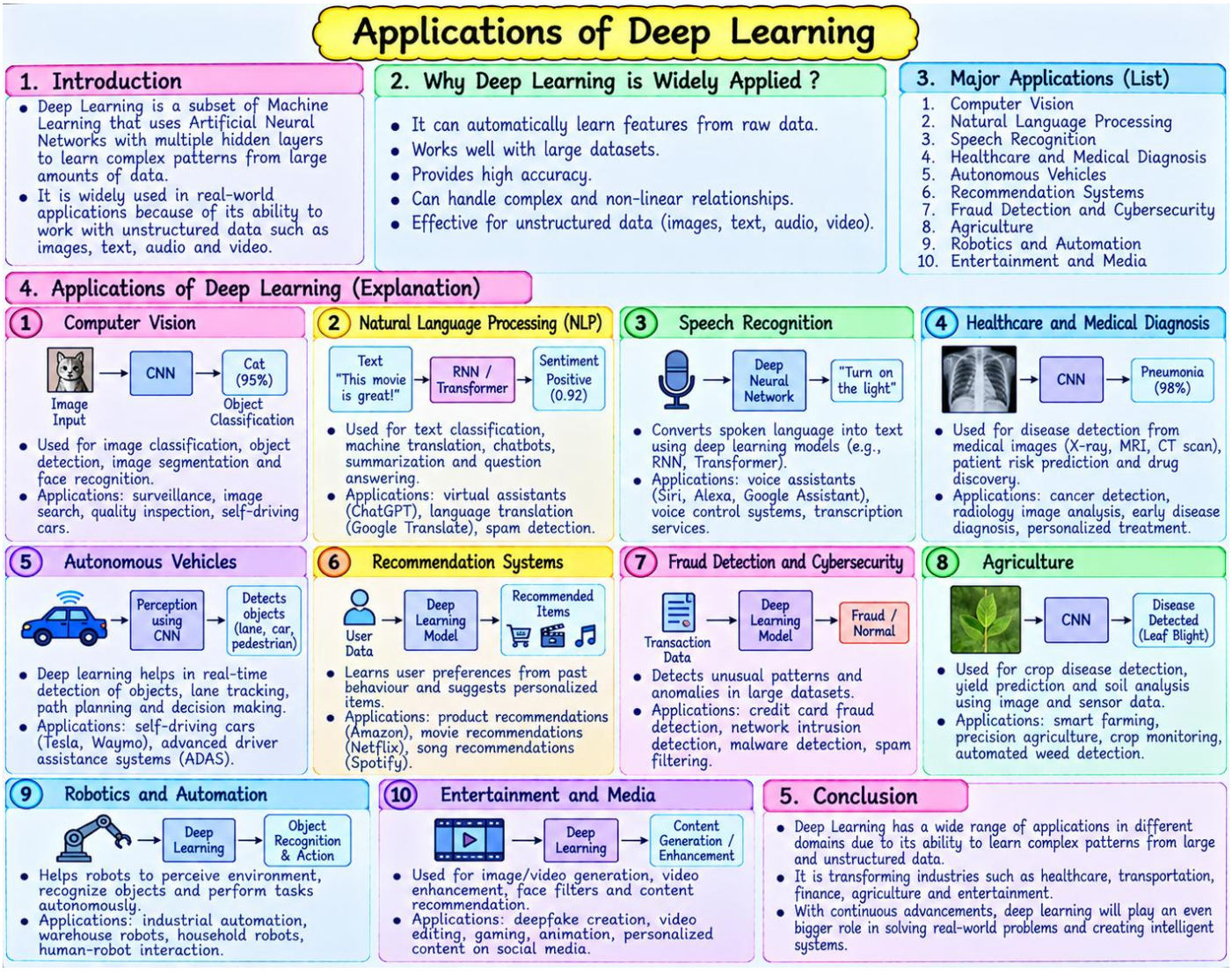
Q2. Discuss different activation function used in Neural Network.

ANS



Q3. List down different applications of Deep Learning. And explain it

ANS



Q4. Describe the role of utility and expected utility in decision theory with example.

ANS

Role of Utility and Expected Utility in Decision Theory with Example

1. Decision Theory (Brief)

- Decision theory is a framework used by an agent to choose the best action among several possible actions under uncertainty.
- It considers the possible outcomes of each action and selects the action that maximizes the expected utility.
- Utility and expected utility are key concepts in decision theory.

2. Utility

- Utility is a numerical measure of the satisfaction or preference an agent receives from an outcome (state).
- It represents the desirability or value of an outcome.
- Utility values are subjective and are assigned based on the agent's goals and preferences.
- Higher utility value means a more desirable outcome.
- Utility can be positive, negative or zero.

3. Expected Utility

- Expected utility is the average utility of all possible outcomes, weighted by their probabilities.
- It helps in selecting the best action when the outcome is uncertain.
- The action with the highest expected utility is chosen.
- It takes both the probability of an outcome and the utility of that outcome into account.

$$EU(a) = \sum_{i=1}^{n} P(s_i|a) \times U(s_i)$$

$EU(a)$: Expected utility of action a

$P(s_i|a)$: Probability of state s_i given action a

$U(s_i)$: Utility of state s_i

n : Number of possible outcomes

4. Role of Utility in Decision Theory

1. Helps to quantify the preference for different outcomes.
2. Converts real-world outcomes (e.g., profit, loss, comfort, risk) into numerical values.
3. Allows comparison of different outcomes on a common scale.
4. Represents the agent's goals and risk attitude.
5. Used to evaluate and compare the desirability of results of different actions.

5. Role of Expected Utility in Decision Theory

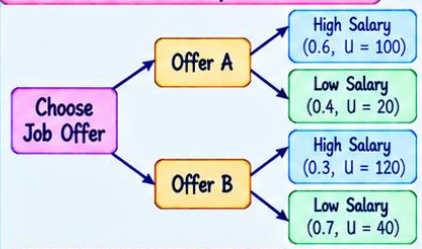
1. Combines the probability of an outcome with its utility.
2. Provides a single numerical value (expected utility) for each action.
3. Helps in choosing the action with the maximum expected utility.
4. Useful when outcomes are uncertain or have multiple possible results.
5. Leads to rational and consistent decision making.
6. Considers both risk and reward in the decision process.

6. Example: Choosing a Job Offer

An agent has two job offers: A and B.
Each offer can result in a High Salary or Low Salary with certain probabilities. The agent assigns utility values based on satisfaction.

Action (Job Offer)	Possible Outcome	Probability	Utility (₹)	Probability × Utility
A	High Salary	0.6	100	0.6 × 100 = 60
	Low Salary	0.4	20	0.4 × 20 = 8
Expected Utility of A				EU(A) = 60 + 8 = 68
B	High Salary	0.3	120	0.3 × 120 = 36
	Low Salary	0.7	40	0.7 × 40 = 28
Expected Utility of B				EU(B) = 36 + 28 = 64

7. Decision Tree Representation



EU(A) = 0.6 × 100 + 0.4 × 20 = 68
EU(B) = 0.3 × 120 + 0.7 × 40 = 64
Since EU(A) > EU(B), the agent should choose Offer A.

8. Interpretation

- Offer A has a higher expected utility (68) than Offer B (64).
- Therefore, according to decision theory, Offer A is the better choice.
- Even though Offer B has a higher possible salary (120), it has a lower probability (0.3), which reduces its expected utility.
- Expected utility helps to balance risk and reward in decision making.

9. Conclusion

- Utility represents the value or desirability of an outcome.
- Expected utility combines the utility of outcomes with their probabilities.
- In decision theory, the action with the highest expected utility is chosen.
- Utility and expected utility help agents make rational decisions under uncertainty by considering both preferences and probabilities.

Q5. Compare and contrast Bagging and Boosting with their application

ANS

Comparison of Bagging and Boosting (10 Points)

Sr. No.	Basis	Bagging (Bootstrap Aggregating)	Boosting
1	Basic Idea	Trains multiple models in parallel using different bootstrap samples and combines their predictions (averaging/voting).	Trains multiple models sequenutially, where each new model focuses on the errors made by previous models.
2	Training Process	Independent training of models.	Sequential training of models.
3	Data Sampling	Uses bootstrap sampling (random samples with replacement).	Uses original data, but gives higher weight to misclassified samples in next iteration.
4	Focus	Reduces variance (helps in avoiding ovenfitting).	Reduces bias (helps in improving accuracy).
5	Type of Learners	Usually uses high variance, unstable models (e.g., Decision Trees).	Uses weak learners (e.g., shallow decision trees) and combines them to make a strong model.
6	Combination Method	Simple averaging (for regression) or majority voting (for classification).	Weighted combination of models (models with better performance get higher weight).
7	Sensitivity to Noise and Outliers	More robust to noise and outliers.	More sensitive to noise and outliers.
8	Overfitting	Less likely to overfit (due to averaging).	Can ovegfit if too many iterations or noisy data.
9	Computational Complexity	Can be trained in parallel, hence faster.	More computationally expensive (sequential training).
10	Examples	Random Forest (Bagging of Decision Trees).	AdaBoost, Gradient Boosting, XGBoost.

Applications of Bagging

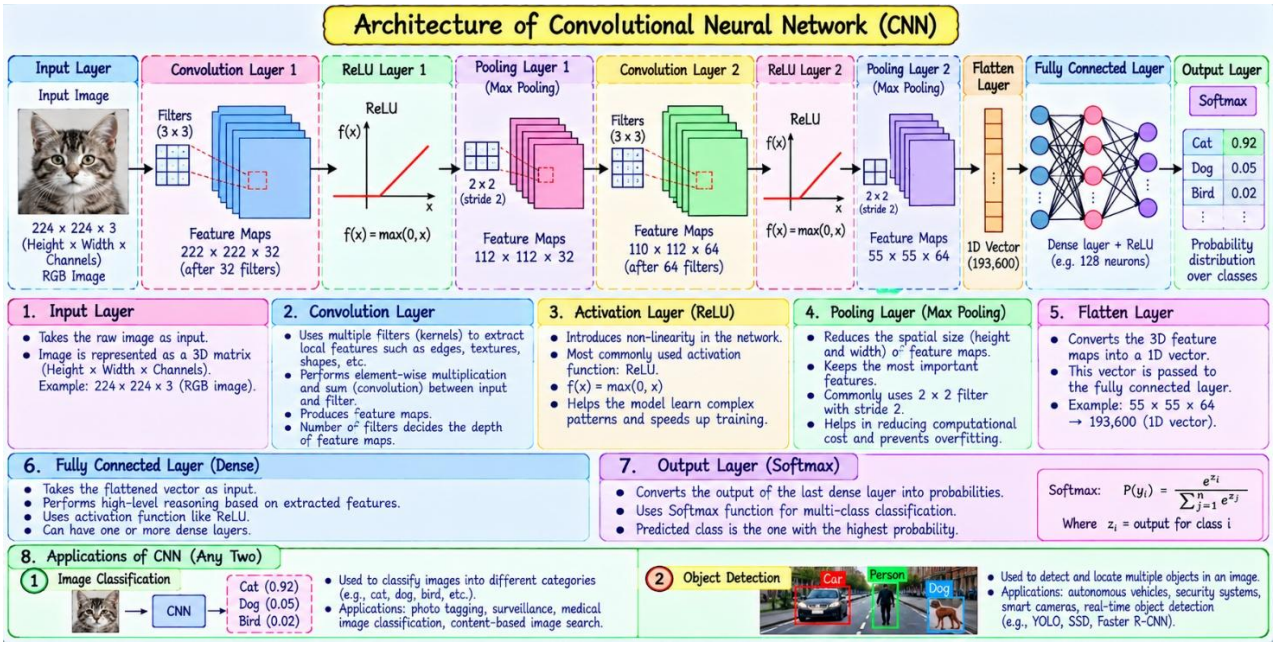
- Random Forest for classification and regression problems.
- Medical diagnosis (disease prediction).
- Image classification (e.g., object recognition).
- Financial risk prediction.
- Useful in problems with high variance and noisy data.

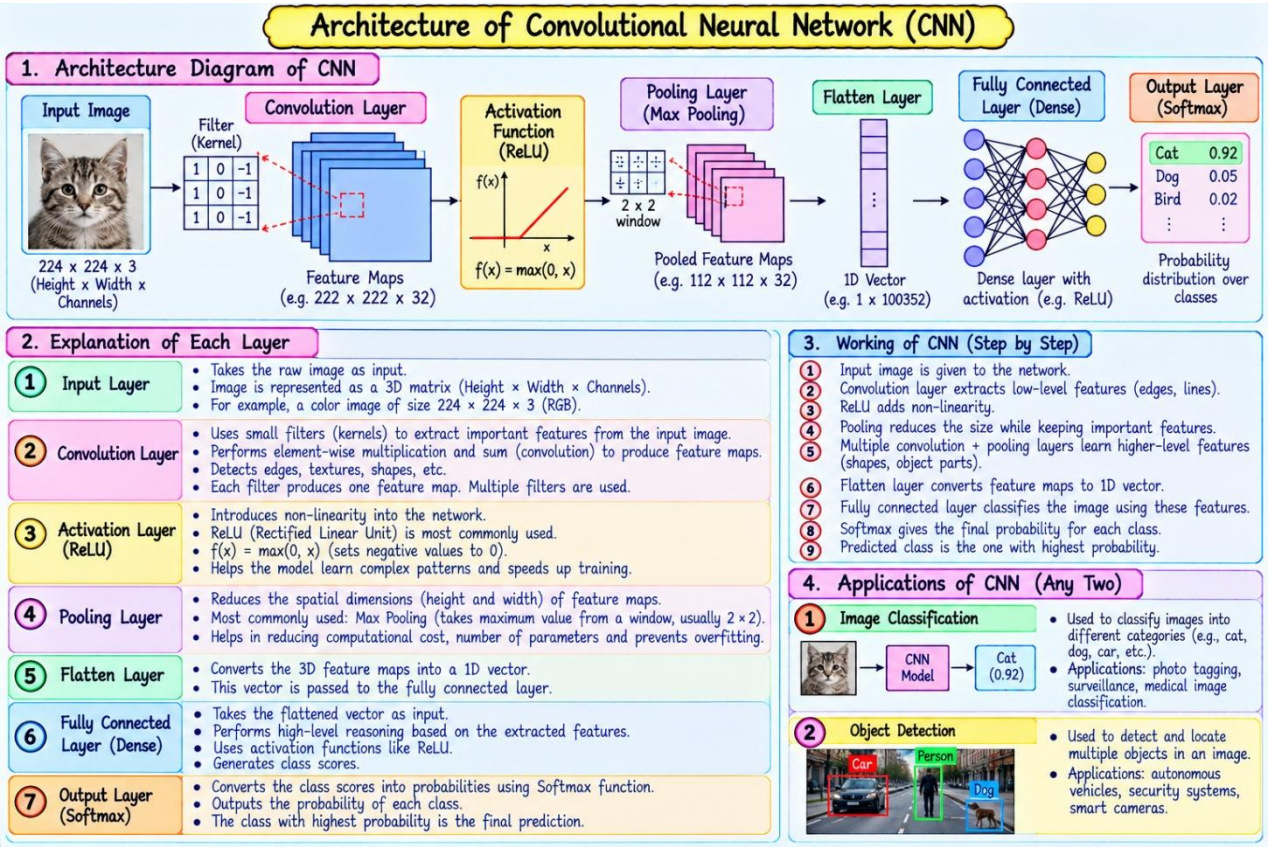
Applications of Boosting

- AdaBoost for classification.
- Gradient Boosting / XGBoost for high accuracy predictions.
- Fraud detection (credit card fraud).
- Recommendation systems.
- Spam detection.
- Useful in complex problems where high predictive accuracy is required.

Q6. Explain Architecture of CNN in detail. List any two applications of it .

ANS





Q.7. What is Bayesian Belief Network? Explain inferencing with example.

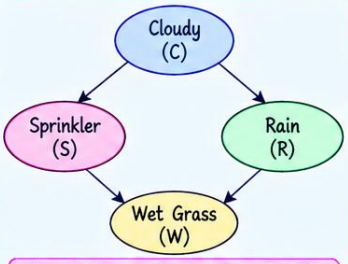
ANS

Bayesian Belief Network (BBN) and Inferencing with Example

1. What is Bayesian Belief Network?

- A Bayesian Belief Network (BBN), also called a Bayesian Network, is a probabilistic graphical model that represents a set of random variables and their conditional dependencies using a directed acyclic graph (DAG).
- Each node represents a random variable (event) and each directed edge represents a causal or dependency relationship.
- Every node has a Conditional Probability Table (CPT) that quantifies the effect of its parent nodes.
- It helps in reasoning under uncertainty by computing the probability of a query variable given some evidence.

2. Structure of a BBN



Directed Acyclic Graph (DAG) of a BBN

3. Components of a BBN

1. **Nodes** : Represent random variables (e.g., Cloudy, Rain).
2. **Directed Edges** : Represent conditional dependencies (causal relationships).
3. **Conditional Probability Tables (CPT)** : Specify the probability distribution of each node given its parents.
4. **Joint Probability Distribution** : Factorized as product of local conditional probabilities.

$$P(X_1, X_2, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid \text{Parents}(X_i))$$

4. Conditional Probability Tables (CPT)

P(C)

Cloudy (C)	P(C)
T	0.5
F	0.5

P(S | C)

C	P(S = T)	P(S = F)
T	0.1	0.9
F	0.5	0.5

P(R | C)

C	P(R = T)	P(R = F)
T	0.8	0.2
F	0.2	0.8

P(W | S, R)

S	R	P(W = T)	P(W = F)
T	T	0.99	0.01
T	F	0.90	0.10
F	T	0.90	0.10
F	F	0.00	1.00

5. Inferencing in BBN

- Inferencing means computing the probability of unknown variables (query variables) given some evidence, using the known network structure and CPTs.
- Common inference tasks:
  1. Prior inference : P(X)
  2. Predictive inference : P(X | evidence)
  3. Diagnostic inference : P(cause | evidence)
  4. Inter-causal inference : P(X | effects)
- Inference can be done using methods like:
  - Enumeration (exact inference)
  - Variable elimination
  - Belief propagation
  - Sampling methods (e.g., Gibbs sampling)
- It uses Bayes' theorem and total probability to compute the required probabilities.

6. Example: Inferencing in BBN

Problem: Given that the grass is wet (W = T), find the probability that it rained (R = T). i.e., P(R = T | W = T).

Solution:

1. Use Bayes' theorem and sum over all possible values of Cloudy (C) and Sprinkler (S).

$$P(R \mid W) = \frac{P(R, W)}{P(W)}$$

2. Compute numerator (using total probability):

$$P(R = T, W = T) = \sum_{C} \sum_{S} P(C)P(S \mid C)P(R \mid C)P(W \mid S, R)$$

3. Substituting values from CPTs:

$$P(R = T, W = T) = 0.353$$

4. Similarly, compute $P(W = T) = 0.647$
5. Finally, $P(R = T \mid W = T) = \frac{0.353}{0.647} = 0.545$

7. Interpretation

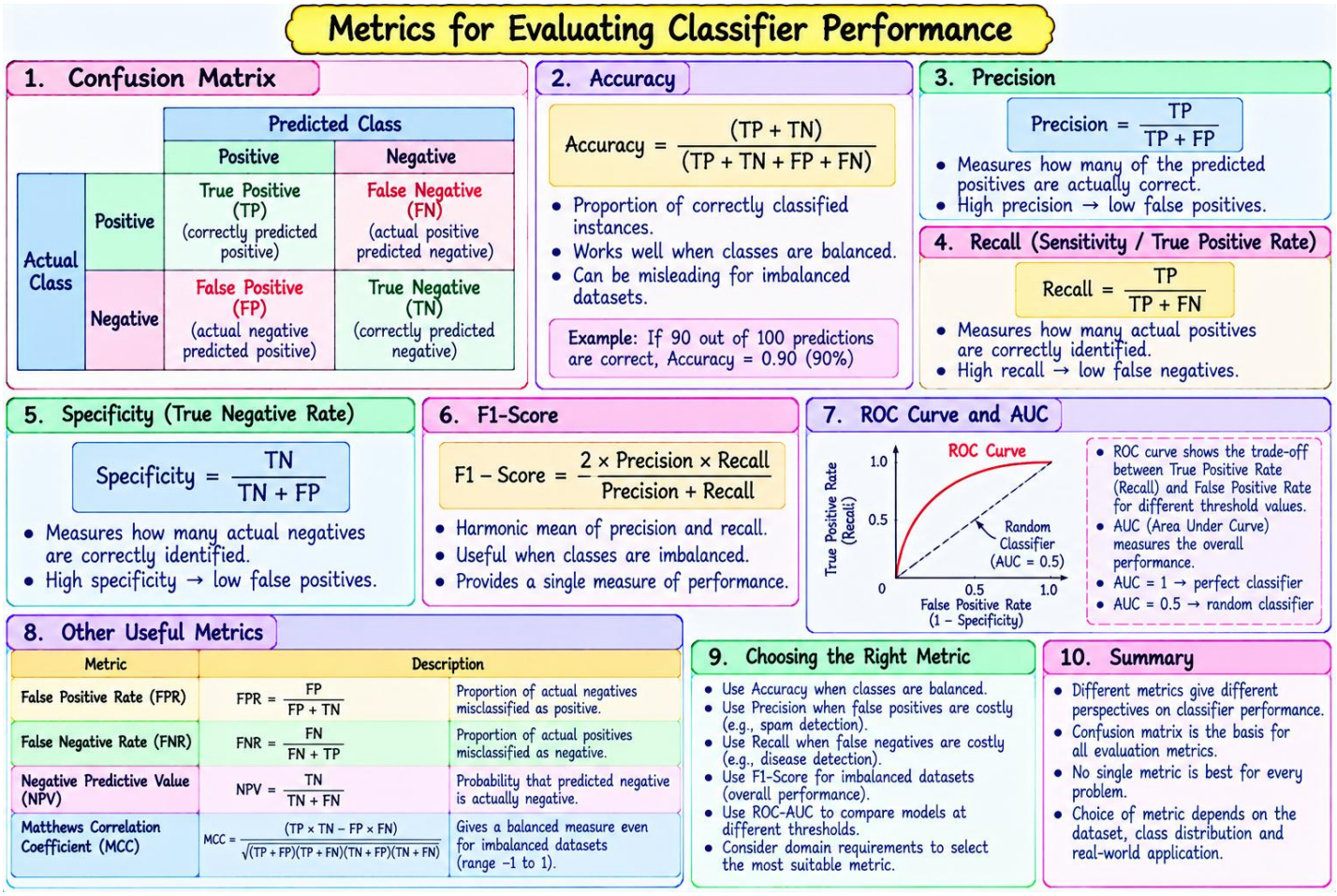
- $P(R = T \mid W = T) = 0.545$ means there is 54.5% probability that it rained, given that the grass is wet.
- Inference helps in reasoning about hidden or unknown causes using observed evidence.

8. Applications of BBN

- Medical diagnosis (e.g., disease prediction)
- Fault detection in engineering systems
- Risk analysis and decision making
- Natural language processing
- Robotics and sensor fusion
- Predicting uncertain events (e.g., weather prediction)

Q.8 Explain Metrics for Evaluating Classifier Performance.

ANS



UT-1

Q1. .Illustrate usage of taxonomies and ontologies for knowledge representation in cognitive system.

ANS

Usage of Taxonomies and Ontologies for Knowledge Representation in Cognitive Systems

Introduction

In **Artificial Intelligence and Data Science**, a **cognitive system** is an intelligent system that can **understand information, learn from data, reason, and make decisions**.

For a cognitive system to work effectively, knowledge must be represented in a structured form. **Taxonomies and ontologies** are two important methods used for **knowledge representation**.

- **Taxonomy** organizes concepts into a **hierarchical structure**.
- **Ontology** represents concepts, their properties, and the **relationships among them**.

They help cognitive systems understand the meaning and relationships of data rather than treating data as isolated facts.

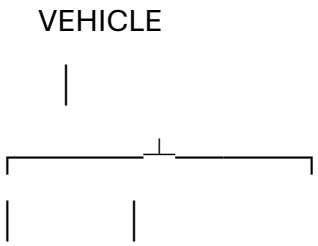
---

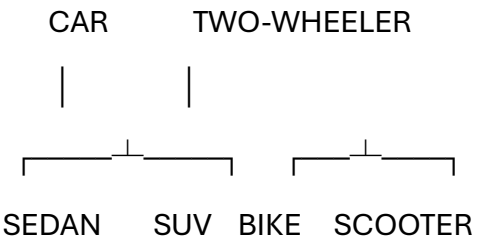
1. Taxonomy

A **taxonomy** is a hierarchical classification of concepts or objects into different categories and subcategories.

It mainly represents an **"is-a" relationship**.

Example:





Here:

- Car **is a** Vehicle.
- Sedan **is a** Car.
- Bike **is a** Two-Wheeler.

Uses of Taxonomy

1. Organizes large amounts of information.
2. Creates a clear hierarchy of concepts.
3. Makes searching and classification easier.
4. Helps in information retrieval.
5. Supports machine learning and data categorization.
6. Helps cognitive systems identify categories and subcategories.

---

2. Ontology

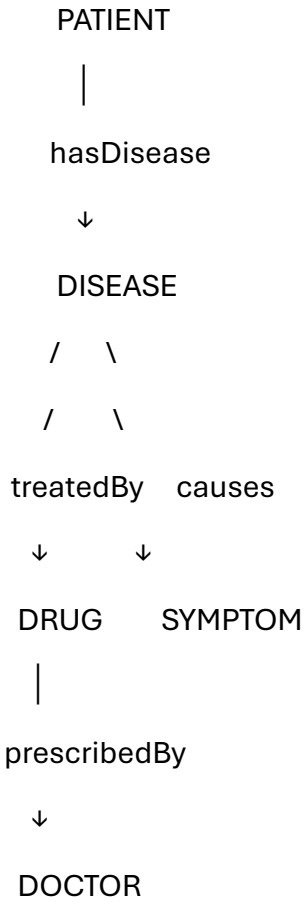
An **ontology** is a formal representation of **concepts, attributes, entities, and relationships** within a particular domain.

Unlike taxonomy, an ontology can represent different types of relationships such as:

- **is-a**
- **has-a**
- **part-of**
- **located-in**
- **causes**
- **used-for**

- **related-to**

Example: Healthcare Ontology



For example:

Patient → hasDisease → Diabetes

Diabetes → treatedBy → Insulin

Diabetes → causes → High Blood Sugar

This allows a cognitive system to understand relationships between different entities.

---

3. Taxonomy vs Ontology

Taxonomy	**Ontology**
Mainly represents hierarchy	Represents concepts and relationships

Taxonomy	Ontology
Mainly uses **is-a** relationship	Uses multiple types of relationships
Simpler structure	More detailed and expressive
Used mainly for classification	Used for reasoning and knowledge representation
Example: Animal → Mammal → Dog	Patient → hasDisease → Diabetes → treatedBy → Insulin

---

4. Usage in Cognitive Systems

Taxonomies and ontologies are used in cognitive systems to organize and understand knowledge.

1. Knowledge Organization

They organize unstructured or complex information into a structured knowledge base.

Example: A healthcare cognitive system can organize information about diseases, symptoms, medicines, and doctors.

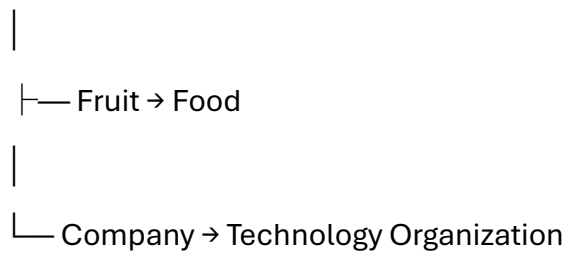
---

2. Semantic Understanding

Ontologies help systems understand the **meaning of data and relationships between concepts**.

For example:

"Apple"



The system can use context and relationships to determine whether **Apple** refers to a fruit or a company.

---

3. Information Retrieval

Taxonomies and ontologies improve search by understanding related concepts.

For example, if a user searches for **“heart disease treatment”**, an ontology can connect:

Heart Disease

↓

Symptoms

↓

Diagnosis

↓

Treatment

↓

Medicines

The system can therefore retrieve more relevant information.

---

4. Reasoning and Decision Making

Ontologies allow cognitive systems to derive **new knowledge from existing facts and relationships**.

Example:

Fact 1: Ravi has Diabetes.

Fact 2: Diabetes is treated by Insulin.

Therefore:

Ravi may require Insulin treatment.

This type of reasoning is useful in intelligent decision-support systems.

---

5. Natural Language Processing (NLP)

Taxonomies and ontologies help NLP systems understand relationships between words and concepts.

They are useful in:

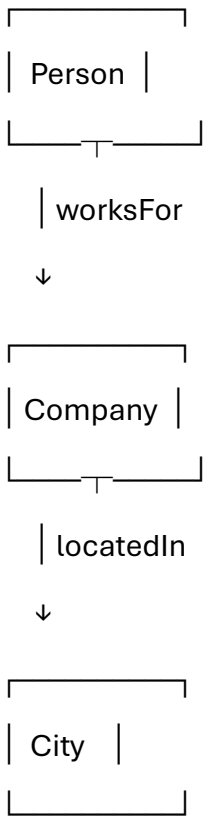
- Chatbots
- Question answering
- Virtual assistants
- Text classification
- Semantic search

---

6. Knowledge Graphs

Ontologies are commonly used to define the structure of **knowledge graphs**.

Example:



The cognitive system can traverse these relationships to answer questions and discover information.

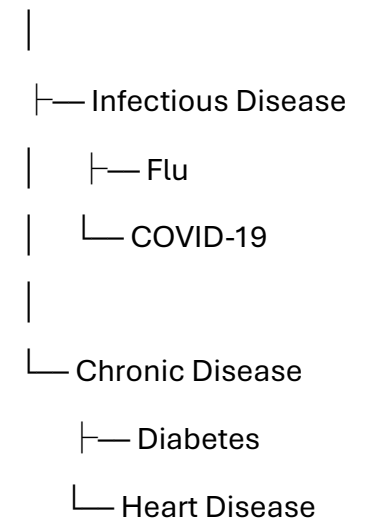
---

7. Example: Cognitive Healthcare System

Consider an AI-based healthcare cognitive system.

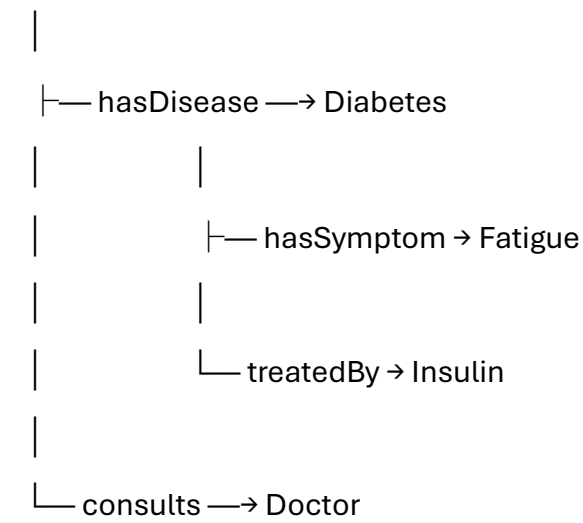
Taxonomy:

Disease



Ontology:

Patient



The **taxonomy** helps classify diseases, while the **ontology** connects patients, diseases, symptoms, treatments, and doctors.

The cognitive system can use this knowledge to:

- Identify possible diseases.
- Find related symptoms.
- Recommend relevant information.
- Support doctors in decision-making.
- Answer patient queries.

---

8. Advantages

1. Provides **structured knowledge representation**.
2. Improves semantic understanding of data.
3. Supports intelligent search and information retrieval.
4. Enables automated reasoning.
5. Improves decision-making.
6. Helps integrate information from different sources.
7. Supports knowledge graphs and NLP applications.
8. Makes cognitive systems more explainable and intelligent.

Conclusion

Taxonomies and ontologies are important techniques for knowledge representation in cognitive systems. A taxonomy provides a **hierarchical classification of concepts**, whereas an ontology provides a richer representation of **concepts, properties, and relationships**. Together, they help cognitive systems **organize knowledge, understand semantics, perform reasoning, retrieve relevant information, and make intelligent decisions**.

⭐ **Exam Summary**

Taxonomy → Classification and hierarchy

Ontology → Concepts + Properties + Relationships + Reasoning

Cognitive System → Understand → Reason → Learn → Decide

This makes taxonomies and ontologies especially useful in **AI, NLP, knowledge graphs, healthcare systems, recommendation systems, and intelligent decision-support systems**.

MEANING OF THIS LINE : AI and DS - 1 (ITC604) , Data Mining & Business Intelligence (ITC601)

The line:

"AI and DS – 1 (ITC604), Data Mining & Business Intelligence (ITC601)"

refers to **two different subjects/courses**, each having its own subject code.

1. AI and DS – 1 (ITC604)

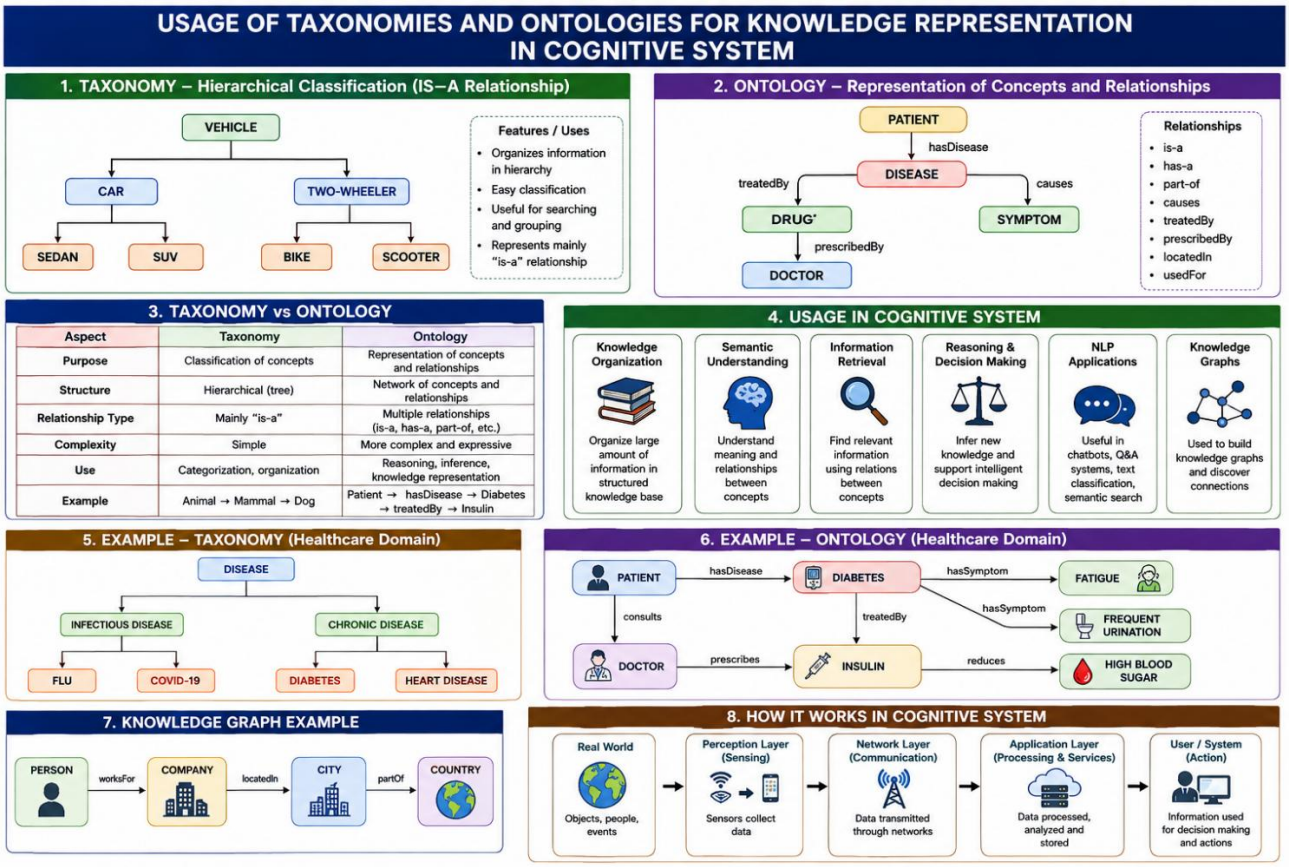
- **AI and DS** = **Artificial Intelligence and Data Science**
- **1** = Part/Subject 1 of the AI and Data Science course
- **ITC604** = The **subject/course code**
- It generally covers topics related to **Artificial Intelligence, Data Science, Machine Learning, cognitive systems, knowledge representation, etc.**

2. Data Mining & Business Intelligence (ITC601)

- **Data Mining** = Finding useful patterns and knowledge from large datasets.
- **Business Intelligence (BI)** = Using data, reports, analytics, and visualization to support **business decision-making**.
- **ITC601** = The **subject/course code**.

Simple Meaning

Subject Code	Subject Name	Main Focus
ITC604	AI and DS – 1	Artificial Intelligence + Data Science
ITC601	Data Mining & Business Intelligence	Data Mining + Business Analytics/Decision Making



Q2. Illustrate inferencing in Bayesian Belief Network with an example.

ANS

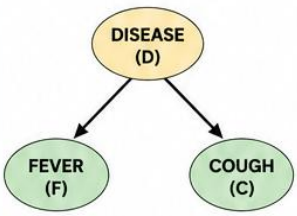
INFERENCING IN BAYESIAN BELIEF NETWORK (BBN) WITH EXAMPLE

1. INTRODUCTION

A Bayesian Belief Network (BBN) is a graphical model that represents uncertain knowledge using nodes (variables), directed edges (dependencies) and Conditional Probability Tables (CPTs).

Inferencing is the process of calculating the probability of unknown events based on observed evidence.

2. BAYESIAN BELIEF NETWORK EXAMPLE



Conditional Probability Tables (CPTs)

P(D)	
D	P(D)
True (Yes)	0.10
False (No)	0.90

P(F | D)	
D	P(F|D)
True	0.80
False	0.20

P(C | D)	
D	P(C|D)
True	0.70
False	0.30

3. PROBLEM (DIAGNOSTIC INFERENCE)

Given Evidence : Patient has Fever (F) and Cough (C)

Find : Probability of Disease given Fever and Cough $P(D \mid F, C)$

Assumption :
Fever (F) and Cough (C) are conditionally independent given Disease (D).

4. INFERENCING CALCULATION USING BAYES' THEOREM

Step 1 : Apply Bayes' Theorem

Bayes' Theorem :

$$P(D|F, C) = \frac{P(F, C \mid D)\, P(D)}{P(F, C)}$$

Since F and C are conditionally independent given D,

$$P(F, C \mid D) = P(F|D) \times P(C|D)$$
$$= 0.8 \times 0.7 = 0.56$$

Step 2 : Probability for Disease

$$P(F, C, D) = P(D) \times P(F|D) \times P(C|D)$$
$$= 0.10 \times 0.8 \times 0.7$$
$$= 0.056$$

Step 3 : Probability for No Disease

$$P(F, C, \neg D) = P(\neg D) \times P(F|\neg D) \times P(C|\neg D)$$
$$= 0.90 \times 0.2 \times 0.3$$
$$= 0.054$$

Step 4 : Total Probability

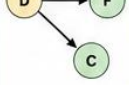
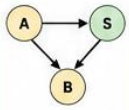
$$P(F, C) = P(F, C, D) + P(F, C, \neg D)$$
$$= 0.056 + 0.054$$
$$= 0.110$$

Step 5 : Posterior Probability (Final Result)

$$P(D|F, C) = \frac{P(F, C, D)}{P(F, C)} = \frac{0.056}{0.110} = 0.509 \ (50.9\%)$$

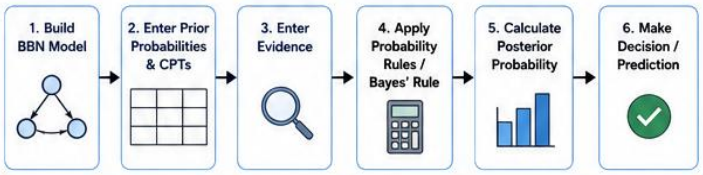
Interpretation : If a patient has both Fever and Cough, the probability of having the Disease (D) is approximately 50.9%.

5. TYPES OF INFERENCING

1. Predictive Inference		From cause to effect. Example : Given Disease (D), find probability of Fever (F).
2. Diagnostic Inference		From effect to cause. Example : Given Fever (F) and Cough (C), infer probability of Disease (D).
3. Intercausal Inference		Between causes through a common effect. Example : Given Symptom (S), information about A affects belief about B.

D = Disease, F = Fever, C = Cough, A and B = Possible causes, S = Symptom (common effect)

6. GENERAL STEPS OF INFERENCING IN BBN



7. APPLICATIONS

- Medical diagnosis (diseases, symptoms)
- Spam detection in emails
- Fault diagnosis in machines
- Weather prediction
- Fraud detection in transactions
- Risk analysis and decision support
- Recommendation systems
- Robotics and autonomous systems

8. ADVANTAGES

- Handles uncertainty and incomplete information
- Provides probabilistic (not just yes/no) answers
- Supports reasoning under uncertainty
- Easy to understand and visualize
- Useful for prediction and decision making
- New evidence can be added and probabilities updated

9. CONCLUSION

Inferencing in Bayesian Belief Network is the process of using known evidence to compute the probability of unknown events.
It uses the structure of the network, prior probabilities, conditional probabilities, and Bayes' theorem to obtain posterior probabilities.
In the given example, observing Fever and Cough changes the probability of Disease to approximately 50.9%.
Thus, Bayesian inferencing is a powerful technique in AI and Data Science for decision-making under uncertainty.

Q3. List and explain the design principles of Cognitive System.

ANS

Design Principles of Cognitive Systems

Introduction

A **Cognitive System** is an intelligent system that can **learn from data, understand information, reason about problems, interact with humans, and make decisions**. Cognitive systems are designed to work with large amounts of structured and unstructured data and continuously improve their performance through learning.

The design principles define the important guidelines that should be followed while developing an effective cognitive system.

Design Principles of Cognitive Systems

1. Data-Driven Design

A cognitive system should be designed to **collect, integrate, and use large amounts of data** from different sources.

- Data may be structured, semi-structured, or unstructured.
- The quality of data directly affects the quality of the system.

Example: A healthcare system uses patient records, medical reports, and sensor data for diagnosis.

---

2. Learning and Adaptation

A cognitive system should be able to **learn from new data and experience**.

- It improves its performance over time.
- Machine Learning and Deep Learning techniques can be used.
- The system should adapt when new information becomes available.

Example: A recommendation system improves its movie recommendations based on the user's previous choices.

---

3. Context Awareness

The system should understand the **context in which information is generated or used**.

Context may include:

- Location
- Time
- User preferences
- Previous activities
- Current situation

Example: A smart assistant gives different recommendations depending on the user's location and time of day.

---

4. Natural Human Interaction

Cognitive systems should allow users to communicate with the system in a **natural and convenient way**.

Interaction can occur through:

- Text
- Speech
- Images
- Gestures
- Natural language

Example: A voice assistant understands a user's spoken question and provides an appropriate answer.

---

5. Reasoning and Decision-Making

A cognitive system should not only store information but also **reason over available knowledge and make decisions**.

It can:

- Compare alternatives

- Identify patterns
- Predict outcomes
- Draw conclusions
- Recommend actions

Example: A medical cognitive system analyzes symptoms and medical history to assist a doctor in diagnosis.

---

6. Knowledge Representation

Knowledge should be represented in a form that allows the system to **understand relationships between concepts**.

Common techniques include:

- Knowledge graphs
- Taxonomies
- Ontologies
- Semantic networks
- Rules

Example:

Patient → hasDisease → Diabetes

Diabetes → treatedBy → Insulin

This allows the system to understand relationships between entities.

---

7. Handling Uncertainty

Real-world information is often **incomplete, noisy, or uncertain**. A cognitive system should therefore be capable of reasoning under uncertainty.

Techniques include:

- Probability
- Bayesian Networks

- Fuzzy Logic
- Probabilistic models

Example: Instead of saying that a patient definitely has a disease, the system may calculate a **75% probability** based on the available symptoms.

---

8. Multimodal Processing

A cognitive system should be capable of processing **different types of information simultaneously**.

It may process:

- Text
- Images
- Audio
- Video
- Sensor data

Example: An autonomous vehicle uses cameras, sensors, GPS, and other data sources to understand its surroundings.

---

9. Explainability and Transparency

The system should provide an understandable explanation of **why it produced a particular result or recommendation**.

This is especially important in:

- Healthcare
- Banking
- Education
- Legal systems
- Safety-critical applications

Example: A loan approval system should explain the major factors that influenced its decision.

---

10. Security and Privacy

A cognitive system must protect **data, models, users, and communications** from unauthorized access.

Important mechanisms include:

- Authentication
- Authorization
- Encryption
- Access control
- Data privacy

Example: A healthcare AI system must protect confidential patient information.

---

11. Scalability and Performance

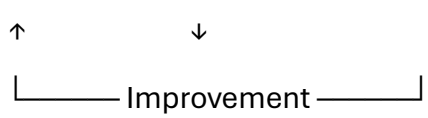
The system should be capable of handling **increasing amounts of data, users, and computational requirements** without major performance degradation.

Example: A banking AI system should continue working efficiently when millions of transactions are processed.

---

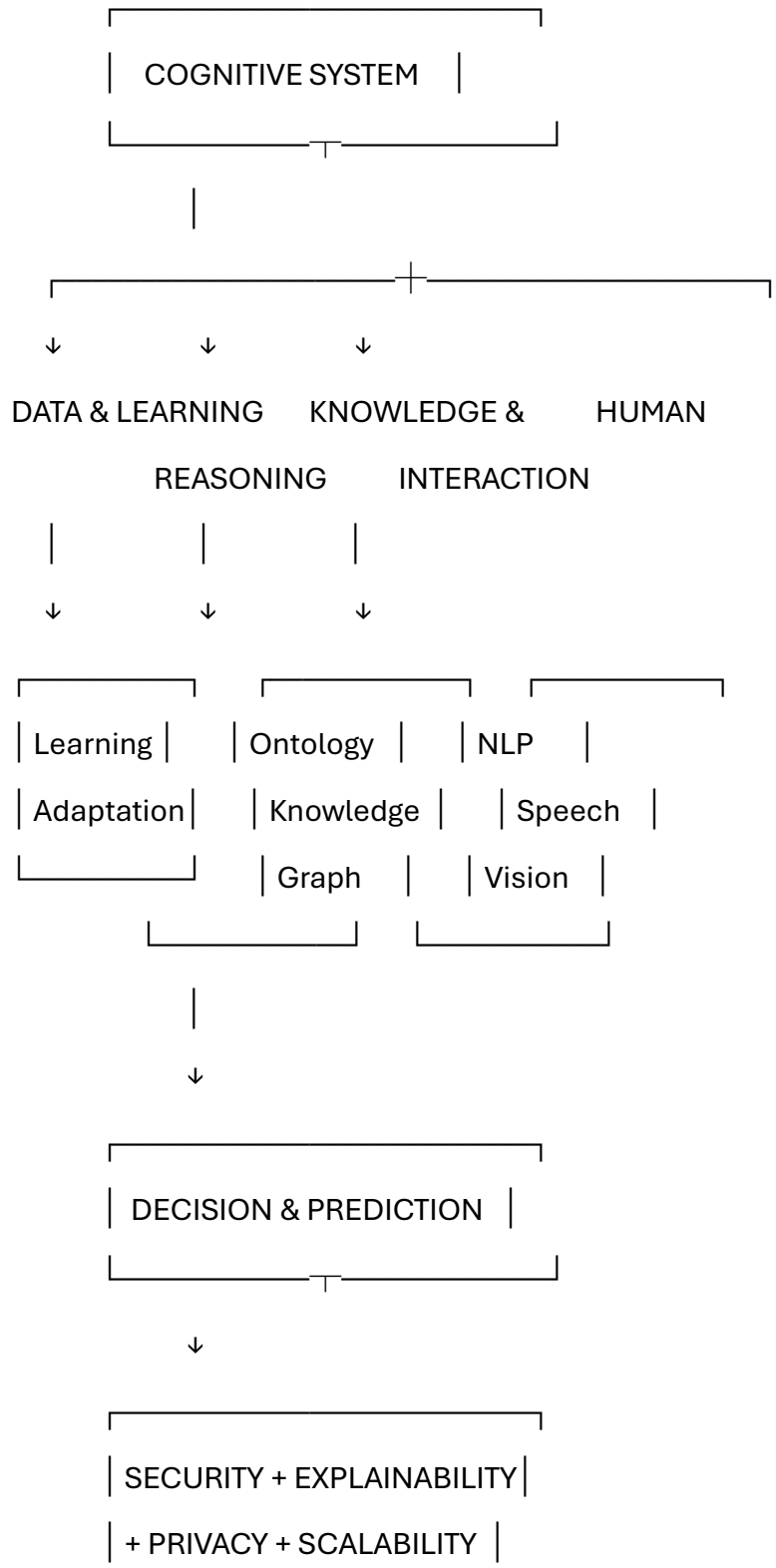
12. Continuous Improvement

A cognitive system should continuously evaluate its performance and **improve its knowledge, models, and decisions** based on new information and feedback.



This makes the system more effective over time.

Simple Diagram of Cognitive System Design Principles



Example: Cognitive Healthcare System

A **cognitive healthcare system** demonstrates these principles:

1. Collects patient data from medical records and sensors.
2. Uses **Machine Learning** to learn from previous cases.
3. Uses an **ontology/knowledge graph** to represent diseases and symptoms.
4. Understands the patient's context and medical history.
5. Uses probabilistic reasoning to estimate possible diseases.
6. Provides a diagnosis or treatment recommendation.
7. Explains the factors behind its recommendation.
8. Protects sensitive patient information.
9. Learns from new medical cases and feedback.

Conclusion

The design of a cognitive system should focus on **learning, adaptation, context awareness, human interaction, reasoning, knowledge representation, uncertainty handling, multimodal processing, explainability, security, privacy, scalability, and continuous improvement**. These principles help create intelligent systems that can **understand information, learn from experience, reason about problems, interact naturally with humans, and make reliable decisions**.

DESIGN PRINCIPLES OF COGNITIVE SYSTEM

1. DATA-DRIVEN DESIGN
Collect, integrate and use large amounts of data from different sources.

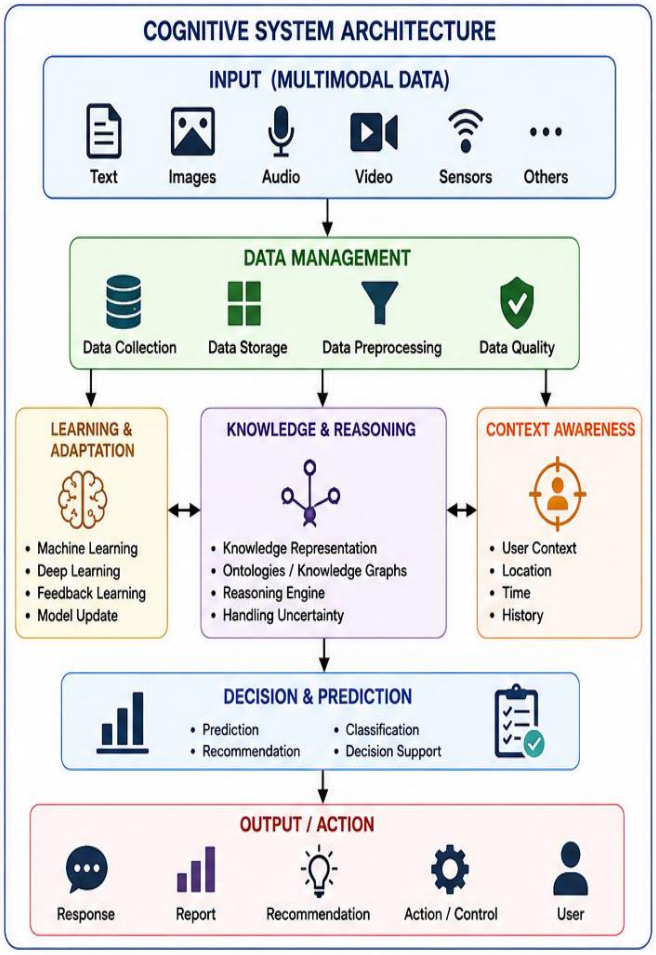
2. LEARNING AND ADAPTATION
System learns from new data and experience and improves its performance over time.

3. CONTEXT AWARENESS
Understand the context such as location, time, user preferences, previous activities and situation.

4. NATURAL HUMAN INTERACTION
Interact with users in natural ways through text, speech, images, gestures and natural language.

5. REASONING AND DECISION-MAKING
Reason over knowledge, identify patterns, predict outcomes and make smart decisions.

6. KNOWLEDGE REPRESENTATION
Represent knowledge in structured form using ontologies, knowledge graphs, taxonomies, rules and semantic networks.



7. HANDLING UNCERTAINTY
Reason under uncertainty using probability, Bayesian networks, fuzzy logic and probabilistic models.

8. MULTIMODAL PROCESSING
Process different types of information like text, image, audio, video and sensor data together.

9. EXPLAINABILITY & TRANSPARENCY
Provide understandable explanations for decisions and recommendations made by the system.

10. SECURITY & PRIVACY
Protect data, models and users using authentication, encryption, access control and privacy policies.

11. SCALABILITY & PERFORMANCE
Handle large scale data, users and computational requirements efficiently and reliably.

12. CONTINUOUS IMPROVEMENT
Continuously evaluate performance and improve models, knowledge, and decisions.



BENEFITS
- ✓ Better decision-making
- ✓ Understand and learn from data
- ✓ Adapt to changing environment
- ✓ Improve user experience
- ✓ Provide reliable and intelligent solutions

Q4. How does Bayeis Theorem determines the probability of an event with uncertain F knowledge? Give ,one example of it.

ANS

BAYES' THEOREM AND DETERMINING PROBABILITY UNDER UNCERTAIN KNOWLEDGE

1. INTRODUCTION

In real world situations, information is often uncertain or incomplete. Bayes' Theorem is used to calculate the probability of an event when new evidence is available.
It helps in updating our belief (probability) about an event.

2. BAYES' THEOREM FORMULA

Bayes' Theorem is given by:

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

Where,

- $P(A|B)$ = Posterior probability (Probability of A given B)
- $P(B|A)$ = Likelihood (Probability of B given A)
- $P(A)$ = Prior probability (Initial probability of A)
- $P(B)$ = Evidence (Total probability of B)

3. HOW BAYES' THEOREM WORKS

Prior Knowledge ($P(A)$)
Initial belief about event A

↓

New Evidence ($P(B)$)
Some new information is observed

↓

Likelihood ($P(B|A)$)
How likely is the evidence if A is true

↓

Apply Bayes' Theorem

↓

Updated Probability ($P(A|B)$)
Posterior probability after considering new evidence

4. EXAMPLE : MEDICAL DIAGNOSIS

Problem:
Find the probability that a person actually has the disease (D) given that the test result is positive (T).

Given:

- $P(D) = 0.01$ (1% people have disease)
- $P(T|D) = 0.90$ (Test positive if disease is present)
- $P(T|\neg D) = 0.05$ (Test positive if disease is absent)

Bayes' Theorem:

$$P(D|T) = \frac{P(T|D)P(D)}{P(T)}$$

5. SOLUTION (STEP BY STEP)

Step 1: Calculate P(T) using Law of Total Probability

$$P(T) = P(T|D)P(D) + P(T|\neg D)P(\neg D)$$

Where, $P(\neg D) = 1 - P(D) = 1 - 0.01 = 0.99$

$$P(T) = (0.90)(0.01) + (0.05)(0.99)$$
$$= 0.009 + 0.0495 = 0.0585$$

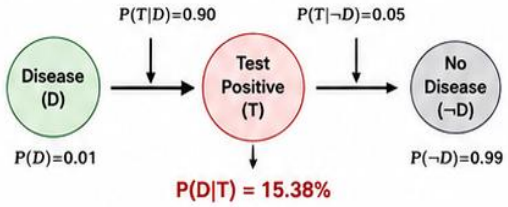
Step 2: Calculate Posterior Probability

$$P(D|T) = \frac{P(T|D)P(D)}{P(T)}$$
$$= \frac{(0.90)(0.01)}{0.0585}$$
$$= \frac{0.009}{0.0585} = 0.1538$$

Therefore, $P(D|T) \approx 0.1538$ or 15.38%

6. INTERPRETATION

Even though the test result is positive, the probability that the person actually has the disease is only 15.38%.
This shows why prior probability and test accuracy both matter when knowledge is uncertain.



7. ANOTHER SIMPLE EXAMPLE : SPAM EMAIL

Given Information	
P(Spam)	= 0.30
P(Word "Offer" | Spam)	= 0.80
P(Word "Offer" | Not Spam)	= 0.10

If an email contains the word "Offer", find P(Spam | Offer)

Using Bayes' Theorem,

$$P(\text{Spam}|\text{Offer}) = \frac{P(\text{Offer}|\text{Spam})P(\text{Spam})}{P(\text{Offer})}$$

$$P(\text{Offer}) = (0.80)(0.30) + (0.10)(0.70)$$
$$= 0.24 + 0.07 = 0.31$$

$$P(\text{Spam}|\text{Offer}) = \frac{(0.80)(0.30)}{0.31} = 0.774$$

So, probability that email is spam given 'Offer' is 77.4%.

8. APPLICATIONS OF BAYES' THEOREM

- **Medical Diagnosis** — Detect disease based on symptoms and test results
- **Spam Detection** — Classify emails as spam or not spam
- **Machine Learning** — Used in Naive Bayes classification
- **Fault Detection** — Identify faults using observed symptoms
- **Weather Prediction** — Update probability of rain or sunshine
- **Fraud Detection** — Detect fraudulent transactions
- **Recommendation Systems** — Predict user preferences
- **Bayesian Networks** — Reasoning among multiple variables

9. ADVANTAGES

- Handles uncertain and incomplete information
- Updates probability with new evidence
- Combines prior knowledge with new observations
- Useful for prediction and decision making
- Widely used in AI and Data Science
- Provides a strong mathematical foundation for reasoning under uncertainty

10. CONCLUSION

Bayes' Theorem helps determine the probability of an event when we have uncertain understanding. It uses prior knowledge, new evidence and likelihood to calculate the updated (posterior) probability. It plays a vital role in AI, machine learning and many real-world intelligent systems.

KEY FORMULA TO REMEMBER

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

Posterior = Updated belief
Likelihood = How likely evidence is
Prior = Initial belief
Evidence = Total probability of evidence

Q5. What is cognitive computing, and how does it differ from traditional computing?

ANS



Cognitive Computing and Its Difference from Traditional Computing

1. Introduction

Cognitive Computing is a branch of **Artificial Intelligence (AI)** that aims to develop computer systems capable of **simulating human cognitive abilities** such as learning, reasoning, understanding natural language, recognizing patterns, and making decisions.

A cognitive computing system can process large amounts of **structured and unstructured data**, learn from experience, understand context, and improve its performance over time.

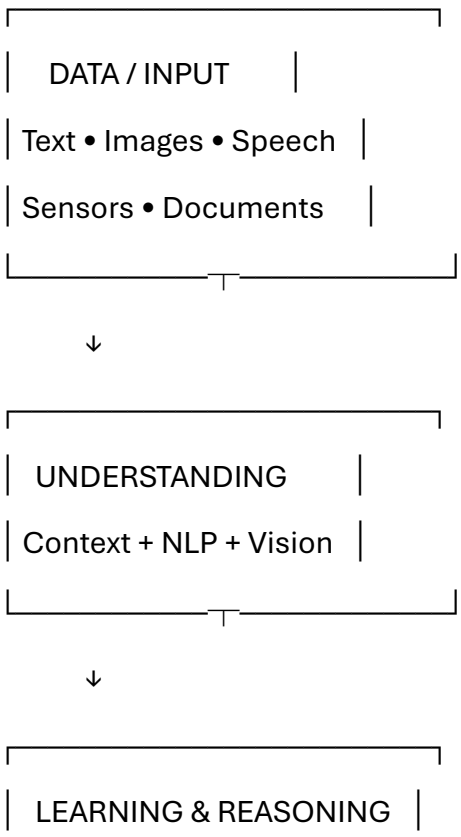
Definition

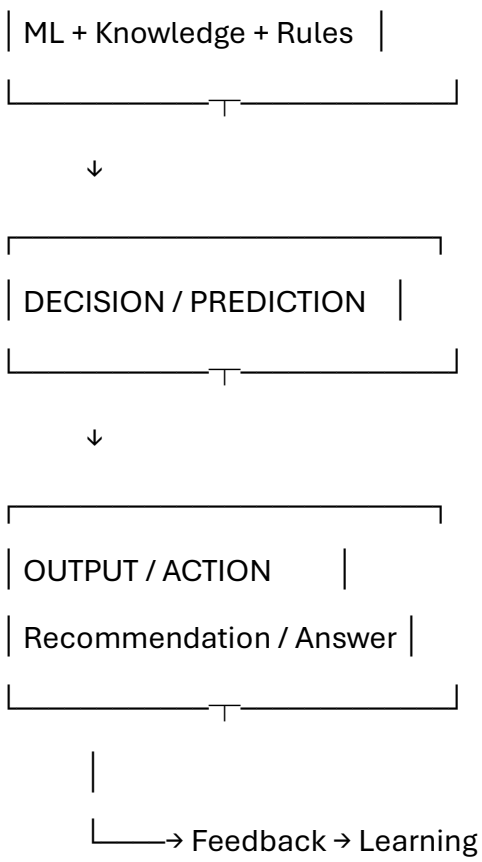
Cognitive Computing is a computing approach that enables systems to learn, reason, understand, interact, and make decisions in a way similar to human intelligence.

Examples: IBM Watson, AI-based medical diagnosis systems, intelligent virtual assistants, recommendation systems, and fraud detection systems.

---

2. Basic Working of Cognitive Computing





---

3. Characteristics of Cognitive Computing

1. Learning

Cognitive systems can **learn from data and previous experiences** and improve their performance.

Example: A recommendation system learns a user's preferences from previous choices.

2. Reasoning

They can analyze available information and **draw conclusions or make decisions**.

Example: A healthcare system can analyze symptoms and suggest possible diseases.

3. Natural Language Understanding

Cognitive systems can understand and process **human language** using Natural Language Processing (NLP).

Example: Chatbots can understand user questions and provide answers.

4. Context Awareness

They consider the **context** of information such as time, location, user preferences, and previous interactions.

5. Handling Uncertainty

Cognitive systems can work with **incomplete, noisy, and uncertain information** using probability and other AI techniques.

6. Multimodal Interaction

They can process different types of data such as:

- Text
- Images
- Audio
- Video
- Sensor data

7. Continuous Improvement

The system can use feedback and new data to **continuously improve its models and decisions**.

---

4. Traditional Computing

Traditional computing is based mainly on **predefined instructions, algorithms, and rules**. The programmer specifies what the system should do for particular inputs.

For example:

Input → Predefined Program/Rules → Output

If the input changes outside the conditions considered by the programmer, the system generally requires **new rules or program modifications**.

Example: A traditional calculator performs mathematical operations according to explicitly programmed instructions.

---

5. Cognitive Computing vs Traditional Computing

Feature	Traditional Computing	Cognitive Computing
Basic approach	Rule/program driven	Learning and knowledge driven
Learning	Does not normally learn automatically	Learns from data and experience
Data	Mainly structured data	Structured + unstructured data
Decision making	Based on predefined rules	Based on learning, reasoning and evidence
Uncertainty	Limited handling	Designed to handle uncertainty
Adaptability	Low	High
Natural language	Limited	Strong NLP capabilities
Context awareness	Usually limited	Can understand context
Human interaction	Command-based	More natural interaction
Output	Usually deterministic	Often probabilistic/recommendation-based
Improvement	Requires programmer changes	Can improve through learning/feedback
Examples	Calculator, payroll system, conventional database application	Virtual assistant, medical AI, recommendation system

---

6. Example to Understand the Difference

Traditional Computing – Loan Approval

A traditional system may use fixed rules:

IF income > ₹50,000

AND credit_score > 750

THEN approve loan

ELSE reject loan

The system follows these predefined rules.

Cognitive Computing – Loan Assessment

A cognitive system can analyze:

- Income
- Credit history
- Previous transactions
- Employment information
- Spending patterns
- Other relevant evidence

It can learn from historical cases and provide a **probability or recommendation** regarding loan risk.

Thus, cognitive computing is more capable of handling **complex and uncertain situations**.

---

7. Applications of Cognitive Computing

Healthcare

- Disease diagnosis
- Patient monitoring
- Medical decision support

Banking and Finance

- Fraud detection
- Credit risk analysis
- Customer support

Education

- Personalized learning

- Intelligent tutoring systems

Retail

- Product recommendations
- Customer behavior analysis

Transportation

- Autonomous vehicles
- Traffic prediction

Customer Service

- Intelligent chatbots
- Virtual assistants

---

8. Advantages of Cognitive Computing

1. Handles large volumes of data.
2. Learns from experience.
3. Supports intelligent decision-making.
4. Handles uncertain and incomplete information.
5. Provides natural human-computer interaction.
6. Can process multiple forms of data.
7. Improves continuously using feedback.
8. Supports personalized services.

Conclusion

Cognitive computing is an AI-based approach that enables computers to **learn, understand, reason, interact, and make decisions** using data and context. Unlike traditional computing, which mainly follows **fixed rules and predefined algorithms**, cognitive computing can **adapt to new information, handle uncertainty, learn from experience, and provide intelligent recommendations**.

Q6. Explain the process of building a cognitive application?

ANS

Process of Building a Cognitive Application

Introduction

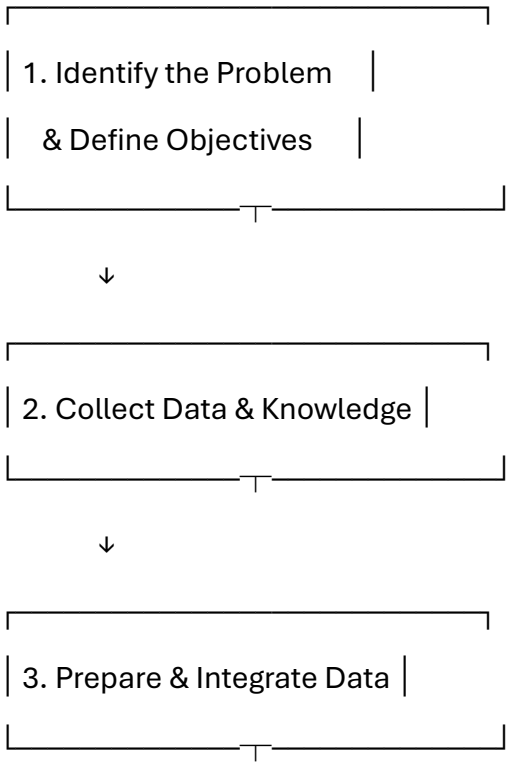
A **Cognitive Application** is an intelligent software application that uses **Artificial Intelligence, Machine Learning, Natural Language Processing, knowledge representation, reasoning, and data analytics** to understand information, learn from experience, and assist users in decision-making.

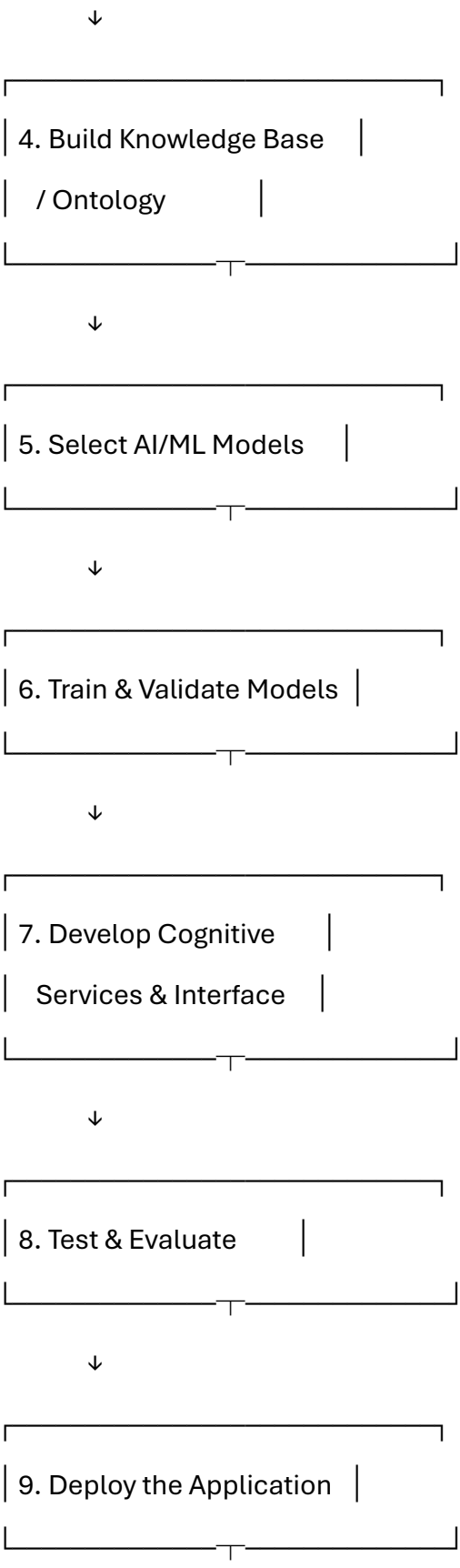
Examples include **AI chatbots, medical diagnosis systems, recommendation systems, intelligent assistants, and fraud detection applications**.

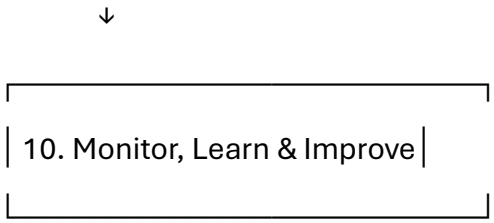
The process of building a cognitive application involves several systematic steps, from identifying the problem to deployment and continuous improvement.

Process of Building a Cognitive Application

Neat Flow Diagram




4. Build Knowledge Base / Ontology
5. Select AI/ML Models
6. Train & Validate Models
7. Develop Cognitive Services & Interface
8. Test & Evaluate
9. Deploy the Application



1. Identify the Problem and Define Objectives

The first step is to clearly define **what problem the cognitive application should solve**.

The developer identifies:

- Target users
- Business or technical problem
- Expected output
- Required decisions or recommendations
- Performance objectives

Example: In healthcare, the objective may be to build an application that helps doctors identify possible diseases from patient symptoms.

---

2. Collect Data and Knowledge

A cognitive application requires sufficient **data and domain knowledge**.

Data can come from:

- Databases
- Sensors
- Documents
- Websites
- Medical records
- Images
- Audio and video
- User interactions

Both **structured and unstructured data** may be required.

Example: A healthcare application may use patient records, symptoms, medical reports, and disease information.

---

3. Prepare and Integrate Data

Raw data is usually incomplete, inconsistent, or noisy. Therefore, it must be prepared before use.

This includes:

- Data cleaning
- Removing duplicates
- Handling missing values
- Data normalization
- Data transformation
- Combining data from different sources

The objective is to create **high-quality and consistent data**.

---

4. Build Knowledge Base and Ontology

A cognitive application needs knowledge about the domain.

Knowledge can be represented using:

- **Taxonomies**
- **Ontologies**
- **Knowledge graphs**
- Rules
- Semantic networks

For example:

Patient

↓ hasDisease

Diabetes

↓ treatedBy

Insulin

This helps the application understand **concepts and relationships** and supports reasoning.

---

5. Select Suitable AI and ML Techniques

Appropriate AI techniques are selected according to the problem.

Examples:

- **Machine Learning** – prediction and classification
- **Deep Learning** – image and speech processing
- **NLP** – understanding text and language
- **Computer Vision** – image/video understanding
- **Bayesian Networks** – reasoning under uncertainty
- **Recommendation algorithms** – personalized suggestions

The choice depends on the type of input, output, data, and application requirements.

---

6. Train and Validate the Models

The selected models are trained using the prepared data.

The general process is:

Training Data → Model Training → Validation → Testing → Model Selection

The model is evaluated using suitable measures such as:

- Accuracy
- Precision
- Recall
- F1-score

- Mean squared error
- Response time

A suitable model is selected based on the application's requirements.

---

7. Develop Cognitive Services and User Interface

The trained models and knowledge base are integrated into the actual application.

Cognitive services may include:

- Speech recognition
- Text understanding
- Image recognition
- Prediction
- Recommendation
- Question answering
- Decision support

The application also needs a suitable **user interface**, such as:

- Web application
- Mobile application
- Chatbot
- Voice assistant
- Dashboard

The interface should allow users to interact naturally with the system.

---

8. Test and Evaluate the Application

The complete application must be tested to ensure that it works correctly.

Testing includes:

- Functional testing

- Model accuracy testing
- Performance testing
- Security testing
- Usability testing
- Reliability testing

The system should be checked with **real-world and unseen data**.

---

9. Deploy the Cognitive Application

After successful testing, the application is deployed for actual use.

It may be deployed on:

- Cloud platforms
- Local servers
- Edge devices
- Mobile devices
- Enterprise systems

Deployment should consider **scalability, security, availability, and response time**.

---

10. Monitor, Learn and Improve

A cognitive application should not remain static. It must continuously **learn from new data and user feedback**.

The improvement cycle is:

New Data

↓

Monitor Performance

↓

Collect Feedback

↓

Update Knowledge / Models

↓

Retrain

↓

Improve Application

This enables the application to adapt to **changing users, new information, and changing environments**.

Example: Cognitive Healthcare Application

Consider an AI-based medical assistant.

Process:

1. **Problem:** Assist doctors in identifying possible diseases.
2. **Data:** Collect symptoms, patient history, reports, and medical images.
3. **Data Preparation:** Clean and standardize medical data.
4. **Knowledge Base:** Build disease-symptom-treatment relationships.
5. **AI Model:** Use Machine Learning and NLP.
6. **Training:** Train the model using historical medical cases.
7. **Application:** Develop a doctor dashboard.
8. **Testing:** Evaluate prediction accuracy and usability.
9. **Deployment:** Deploy in hospitals or clinics.
10. **Improvement:** Use new cases and feedback to update the system.

Working:

Patient Data

↓

Data Processing

↓

Knowledge Base + AI/ML Model

↓

Reasoning / Prediction

↓

Possible Diagnosis

↓

Doctor Decision

↓

Feedback → Continuous Learning

Important Components of a Cognitive Application

A typical cognitive application contains:

Data Sources + Knowledge Base + AI/ML Models + Reasoning Engine + Cognitive Services + User Interface + Feedback Mechanism

These components work together to make the application **intelligent, adaptive, and decision-oriented**.

Advantages of This Process

1. Provides a systematic approach to application development.
2. Improves data quality and model accuracy.
3. Supports intelligent decision-making.
4. Enables natural interaction with users.
5. Allows continuous learning and adaptation.
6. Supports large-scale and real-world applications.
7. Improves efficiency and personalization.

Conclusion

Building a cognitive application is a **continuous and iterative process**. It starts with identifying the problem and collecting data, followed by data preparation, knowledge

representation, AI model development, testing, deployment, and continuous improvement. The final application combines **data, knowledge, AI, reasoning, and user interaction** to provide intelligent and useful services.

PROCESS OF BUILDING A COGNITIVE APPLICATION

1 IDENTIFY THE PROBLEM AND DEFINE OBJECTIVES

- Understand the problem domain and user needs.
- Define goals, expected outcomes and success criteria.

2 COLLECT DATA AND KNOWLEDGE

- Gather data from multiple sources (structured and unstructured).
- Collect domain knowledge from experts, documents, databases etc.

3 PREPARE AND INTEGRATE DATA

- Clean, transform and normalize data.
- Handle missing values, remove duplicates.
- Integrate data from different sources.

4 BUILD KNOWLEDGE BASE / ONTOLOGY

- Represent domain knowledge using ontologies, taxonomies, rules or knowledge graphs.
- Define entities, relationships and concepts.

5 SELECT AI / ML TECHNIQUES

- Choose suitable techniques based on the problem.
- Examples: ML, Deep Learning, NLP, Computer Vision, Bayesian Networks.

6 TRAIN AND VALIDATE MODELS

- Train models using prepared data.
- Validate and test models.
- Evaluate using accuracy, precision, recall and other metrics.

7 DEVELOP COGNITIVE SERVICES AND USER INTERFACE

- Integrate models and knowledge base into services (APIs).
- Build UI for users (web, mobile, chatbot, voice assistant, dashboard).

8 TEST AND EVALUATE

- Perform functional, performance, security and usability testing.
- Check with real-world and unseen data.

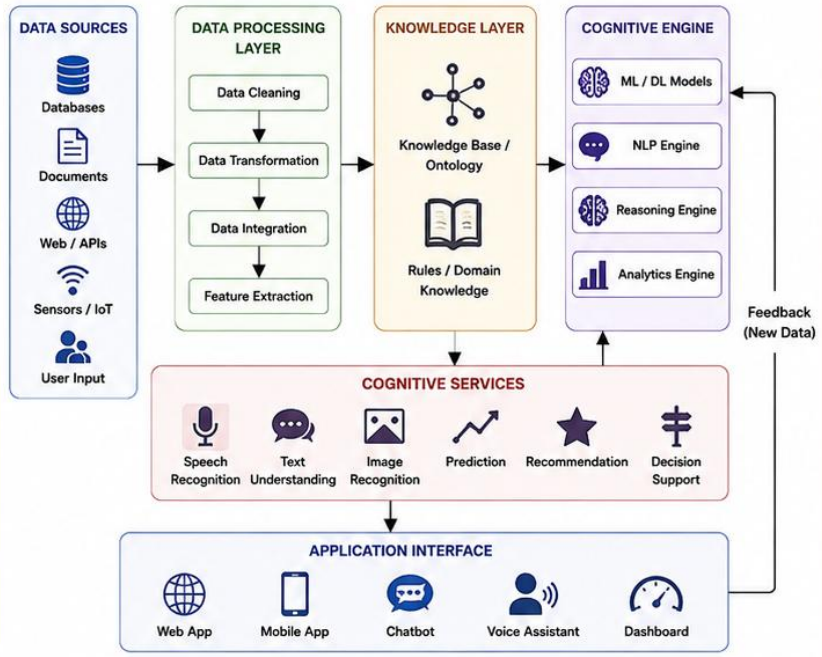
9 DEPLOY THE APPLICATION

- Deploy on cloud, servers or edge devices.
- Ensure scalability, availability and security.

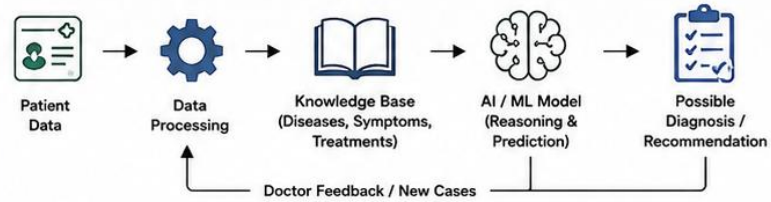
10 MONITOR, LEARN AND IMPROVE

- Monitor performance and user feedback.
- Collect new data and update models.
- Continuously improve the application.

ARCHITECTURE OF A COGNITIVE APPLICATION



EXAMPLE : COGNITIVE HEALTHCARE APPLICATION



CONTINUOUS LEARNING AND IMPROVEMENT



BENEFITS OF COGNITIVE APPLICATIONS



Q7. Describe Natural Language Processing in support of Cognitive System.

ANS

Natural Language Processing (NLP) in Support of Cognitive Systems

1. Introduction

Natural Language Processing (NLP) is a branch of **Artificial Intelligence (AI)** that enables computers to **understand, interpret, process, and generate human language**.

In a **Cognitive System**, NLP acts as an important component for **human-computer interaction**. It allows the cognitive system to understand information in the form of **text and speech**, learn from it, reason about it, and provide meaningful responses.

Examples: Chatbots, virtual assistants, medical AI assistants, search engines, and question-answering systems.

---

2. Definition

Natural Language Processing is a technology that enables computers to understand, analyze, interpret, and generate human language in text or speech form.

It supports cognitive systems by converting human language into **machine-understandable information** and generating responses that are understandable to humans.

---

3. Role of NLP in Cognitive Systems

A cognitive system generally works as:

Human Language

↓

NLP

↓

Understand Meaning

↓

Knowledge / Reasoning

↓

Decision / Response

↓

Human

NLP provides the communication link between the **human user and the cognitive system**.

---

4. Major NLP Processes

1. Text Acquisition

The system first obtains language data from:

- Documents
- Emails
- Websites
- Chats
- Social media
- User queries

For speech-based systems, voice is first converted into a usable digital representation.

---

2. Tokenization

Tokenization divides a sentence into smaller units called **tokens**, usually words or phrases.

Example:

"AI helps doctors."

↓

AI | helps | doctors

This makes the text easier for the system to process.

---

3. Part-of-Speech Tagging

NLP identifies the grammatical role of words.

Example:

"The doctor treats patients."

The → Determiner

doctor → Noun

treats → Verb

patients → Noun

This helps the system understand sentence structure.

---

4. Named Entity Recognition (NER)

NER identifies important entities in text such as:

- Person
- Organization
- Location
- Date
- Medical condition
- Product

Example:

"Dr. Ravi works at Mumbai Hospital."

Dr. Ravi → Person

Mumbai Hospital → Organization

Mumbai → Location

---

5. Syntactic Analysis

It determines the **grammatical structure and relationships** among words in a sentence.

For example, it identifies the **subject, verb, and object**.

"Student reads book."

Student → Subject

reads → Verb

book → Object

---

6. Semantic Analysis

Semantic analysis determines the **meaning of the sentence**.

Example:

"Apple is good for health."

The system identifies **Apple as a fruit**, rather than a technology company, using context.

---

7. Sentiment Analysis

It identifies the **emotion or opinion** expressed in text.

Examples:

- Positive
- Negative
- Neutral

Example:

“The service is excellent.”

Output → **Positive**

---

8. Question Answering

NLP enables cognitive systems to understand user questions and generate suitable answers.

Example:

User:

"What is the capital of India?"

Cognitive System:

"New Delhi."

This is widely used in **chatbots and virtual assistants**.

---

9. Language Generation

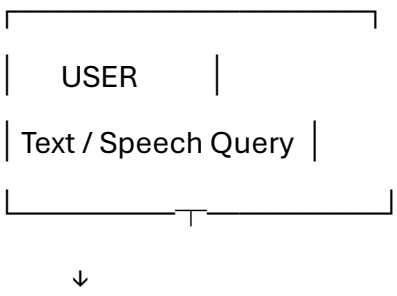
NLP can generate human-readable text from processed information.

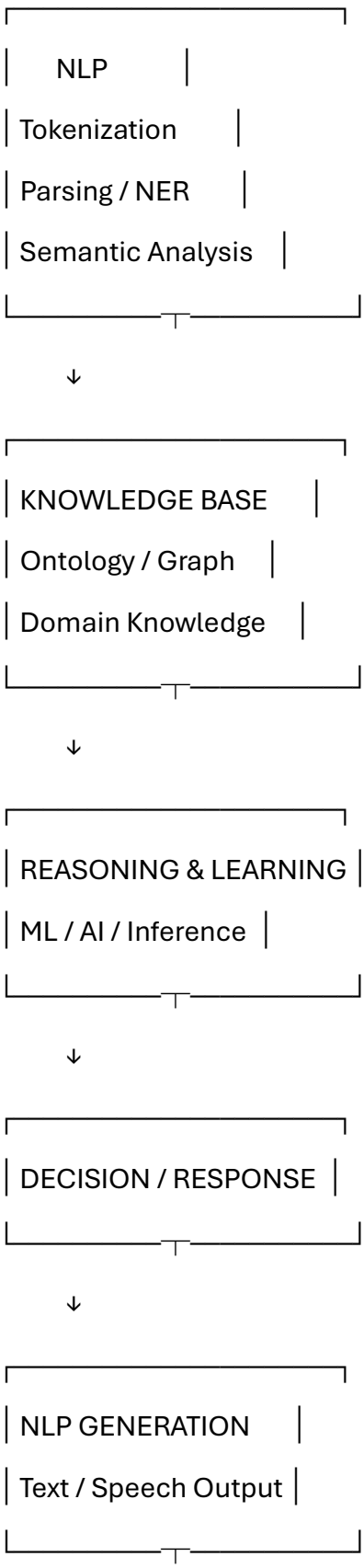
Applications include:

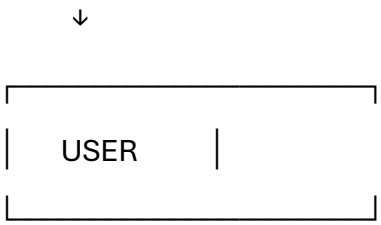
- Chatbot responses
- Reports
- Summaries
- Recommendations
- Automated emails

---

5. NLP and Cognitive System Architecture




NLP
Tokenization
Parsing / NER
Semantic Analysis
KNOWLEDGE BASE
Ontology / Graph
Domain Knowledge
REASONING & LEARNING
ML / AI / Inference
DECISION / RESPONSE
NLP GENERATION
Text / Speech Output



---

6. How NLP Supports Cognitive Systems

1. Natural Human Interaction

NLP allows users to interact with cognitive applications using **normal language instead of programming commands**.

Example: A user can ask a chatbot, “What is my account balance?”

---

2. Understanding Unstructured Data

A large amount of information exists in the form of **documents, emails, articles, reports, and conversations**.

NLP converts this unstructured information into useful knowledge.

---

3. Knowledge Extraction

NLP can extract important facts, entities, and relationships from large text collections.

Example:

"Diabetes is treated with insulin."

Diabetes → treatedBy → Insulin

This information can be stored in a **knowledge graph or ontology**.

---

4. Reasoning and Decision Support

After extracting knowledge, the cognitive system can combine it with existing knowledge to support decisions.

Example: In healthcare, NLP can extract symptoms from a patient's report and provide relevant information to a doctor.

---

5. Personalized Responses

NLP helps cognitive systems understand user queries and provide responses according to the **user's context and requirements**.

---

6. Continuous Learning

NLP systems can learn from:

- New documents
- User conversations
- Feedback
- New terminology

This helps cognitive applications improve continuously.

---

7. Example: Cognitive Healthcare System

Suppose a patient enters:

“I have fever, cough, and difficulty breathing.”

The NLP system processes the sentence:

Input

↓

Text Processing

↓

Entity & Symptom Extraction

↓

Fever + Cough + Breathing Difficulty

↓

Knowledge Base / Medical Ontology

↓

AI Reasoning

↓

Possible Conditions / Recommendation

↓

Doctor / Patient Response

Here, NLP converts the patient's natural-language statement into **structured information** that the cognitive system can use for reasoning and decision support.

---

8. Applications of NLP in Cognitive Systems

NLP is used in:

1. **Chatbots and virtual assistants**
2. **Medical diagnosis and clinical decision support**
3. **Search engines and semantic search**
4. **Question-answering systems**
5. **Sentiment analysis**
6. **Text summarization**
7. **Machine translation**
8. **Fraud and risk analysis**
9. **Customer support**
10. **Intelligent tutoring systems**

---

9. Advantages of NLP in Cognitive Systems

1. Enables **natural human-computer interaction**.
2. Processes large volumes of text and speech.

3. Extracts useful information from unstructured data.
4. Supports intelligent search and question answering.
5. Helps cognitive systems understand user intent.
6. Supports decision-making and reasoning.
7. Provides personalized responses.
8. Reduces manual processing of language-based information.

Conclusion

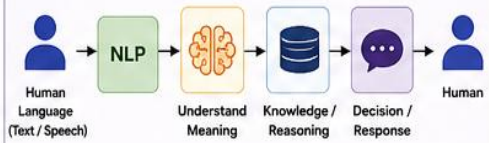
Natural Language Processing is a key technology for Cognitive Systems because it enables machines to communicate with humans using natural language. NLP helps a cognitive system **understand text and speech, extract knowledge, identify context, reason about information, answer questions, and generate meaningful responses**.

NATURAL LANGUAGE PROCESSING (NLP) IN SUPPORT OF COGNITIVE SYSTEM

1. INTRODUCTION

Natural Language Processing (NLP) is a branch of AI that enables computers to understand, interpret, process and generate human language (text or speech). In a cognitive system, NLP acts as a bridge between human and machine.

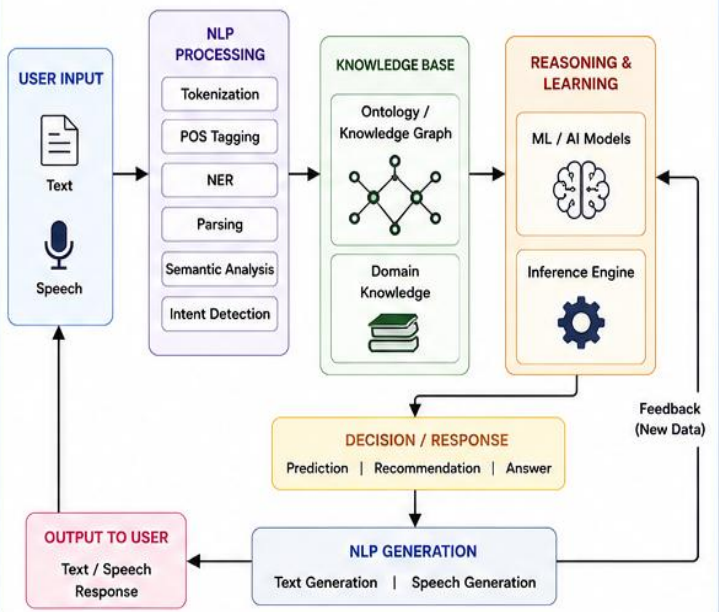
2. ROLE OF NLP IN COGNITIVE SYSTEMS



3. MAJOR NLP PROCESSES

1 Text Acquisition	Get data from various sources (text, speech, documents etc.)	Documents, Web Pages, Speech
2 Tokenization	Split sentences into tokens (words or phrases).	"AI helps doctors." → AI | helps | doctors | .
3 Part-of-Speech Tagging	Identify grammatical role of each word.	The/Det doctor/N treats/V patients/N
4 Named Entity Recognition (NER)	Identify entities like person, organization, location, etc.	Dr. Ravi works at Mumbai Hospital. (Person, Organization, Location)
5 Syntactic Analysis (Parsing)	Determine grammatical structure and relations.	Student reads book. (Subject, Verb, Object)
6 Semantic Analysis	Determine meaning of sentence using context.	"Apple is good for health." (Apple → fruit)
7 Sentiment Analysis	Identify emotion or opinion in the text.	"The service is excellent." → Positive
8 Question Answering	Understand user question and generate appropriate answer.	Q: Capital of India? A: New Delhi.
9 Language Generation	Generate human readable text from processed info.	Summary, Report, Chatbot Response, etc.

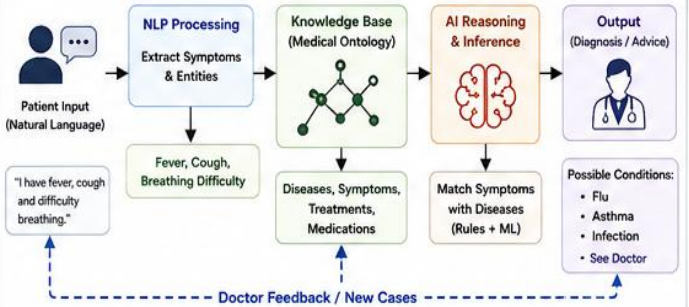
4. NLP AND COGNITIVE SYSTEM ARCHITECTURE



5. HOW NLP SUPPORTS COGNITIVE SYSTEMS

- **Natural Human Interaction** — Allows users to communicate using normal language.
- **Understanding Unstructured Data** — Converts large unstructured text into structured and useful information.
- **Knowledge Extraction** — Extracts entities, facts, relationships and events from text.
- **Reasoning & Decision Support** — Provides meaningful information to support reasoning and decisions.
- **Personalized Responses** — Understands user intent and context to provide relevant responses.
- **Continuous Learning** — Learns from new data, feedback and improves performance over time.

6. EXAMPLE: COGNITIVE HEALTHCARE SYSTEM



7. APPLICATIONS OF NLP IN COGNITIVE SYSTEMS



8. ADVANTAGES OF NLP IN COGNITIVE SYSTEMS



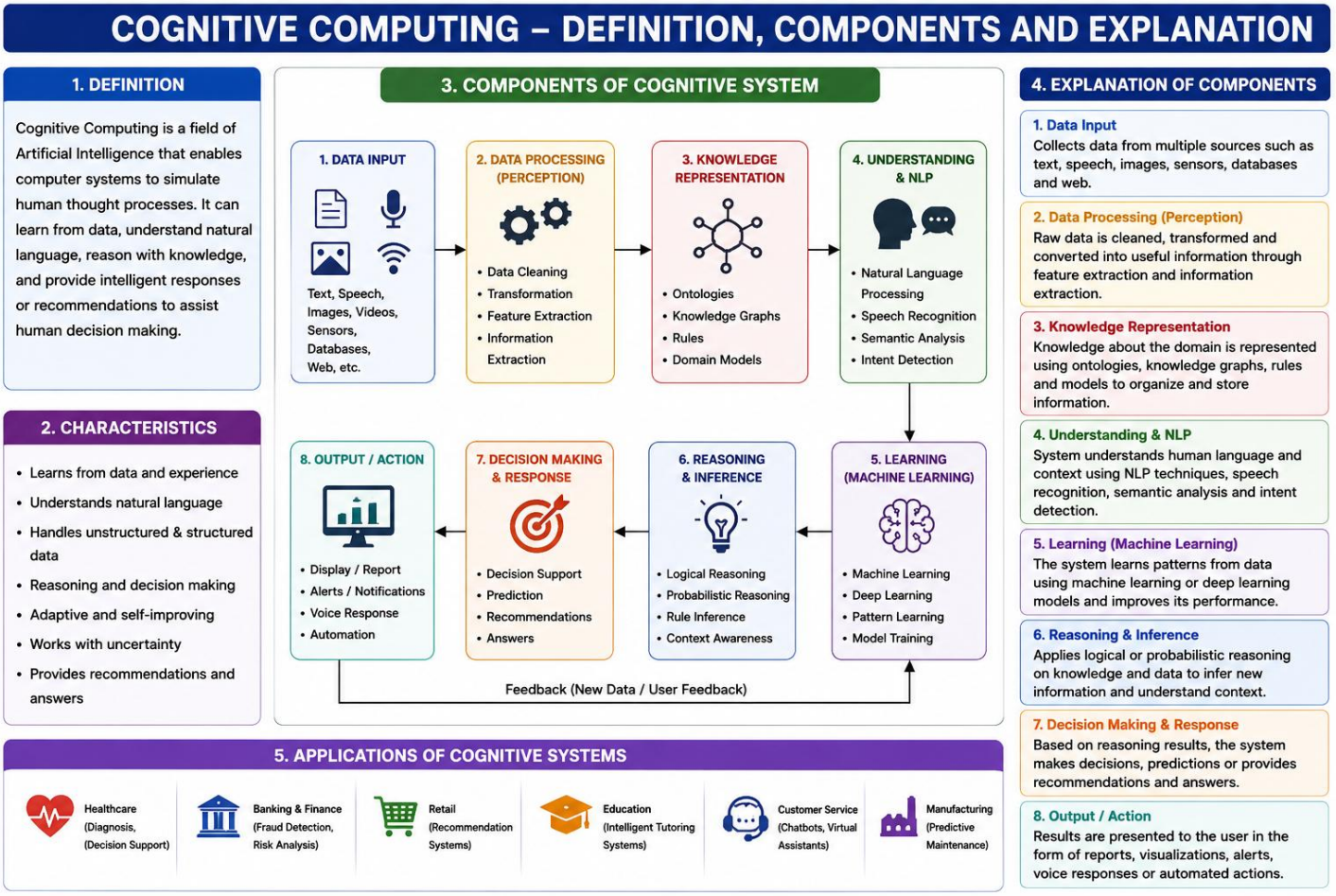
9. CONCLUSION

NLP is a key technology that enables cognitive systems to understand human language. It helps in extracting knowledge, understanding context, reasoning over information and generating useful responses. By integrating NLP with knowledge, learning and reasoning, cognitive systems become intelligent, adaptive and user-friendly.



Q8. Define Cognitive Computing.Draw a neat diagram of Components of the Cognitive Sytem and explain the components.

ANS



Cognitive Computing – Definition and Components of a Cognitive System

1. Definition of Cognitive Computing

Cognitive Computing is a branch of **Artificial Intelligence (AI)** that enables computer systems to **learn from data, understand natural language, recognize patterns, reason with knowledge, and assist humans in decision-making**.

In simple words:

Cognitive computing makes computers more capable of learning, understanding, reasoning, and responding like humans.

Examples: AI medical diagnosis systems, intelligent chatbots, virtual assistants, recommendation systems, and fraud detection systems.

---

2. Neat Diagram – Components of Cognitive System

The **image above** shows the complete **components and information flow of a Cognitive System**.

Basic Flow

Data Input → Data Processing → Knowledge Representation → Understanding/NLP → Learning → Reasoning → Decision Making → Output/Action → Feedback

---

3. Components of a Cognitive System

1. Data Input

This is the starting point of a cognitive system. It collects information from different sources.

Sources include:

- Text
- Speech
- Images
- Videos
- Sensors
- Databases
- Websites

Example: A healthcare system receives patient records, medical reports, and symptoms.

---

2. Data Processing / Perception

The collected data is usually raw and needs to be processed before it can be used.

Main functions:

- Data cleaning
- Data transformation
- Feature extraction
- Information extraction

Example: Removing unnecessary information from a medical report and extracting important symptoms.

---

3. Knowledge Representation

Knowledge about a particular domain is stored in a structured form so that the system can understand and reason over it.

Techniques include:

- Ontologies
- Knowledge graphs
- Rules
- Taxonomies
- Domain models

Example:

Diabetes → treatedBy → Insulin

This represents the relationship between a disease and its treatment.

---

4. Understanding and Natural Language Processing (NLP)

This component enables the system to **understand human language**.

It may perform:

- Natural Language Processing
- Speech recognition
- Semantic analysis
- Intent detection

Example: A chatbot understands the question, *"What are the symptoms of diabetes?"*

---

5. Learning / Machine Learning

The cognitive system learns **patterns and relationships from data** and improves its performance over time.

Techniques include:

- Machine Learning
- Deep Learning
- Pattern recognition
- Model training

Example: A recommendation system learns the user's preferences from previous purchases.

---

6. Reasoning and Inference

This component uses the available **knowledge, evidence, and learned information** to derive conclusions.

It may use:

- Logical reasoning
- Probabilistic reasoning
- Rule-based inference
- Bayesian reasoning
- Context awareness

Example: Based on symptoms and medical knowledge, the system can infer possible diseases.

---

7. Decision Making and Response

After reasoning, the system produces an appropriate **decision, prediction, recommendation, or answer**.

Examples:

- Predicting a disease
- Recommending a product
- Detecting fraud
- Answering a question
- Supporting a business decision

---

8. Output / Action

The final result is presented to the user or used to perform an action.

Forms of output:

- Reports
- Dashboards
- Alerts
- Voice responses
- Recommendations
- Automated actions

Example: A system sends an alert to a doctor when a patient's condition requires attention.

---

4. Feedback and Continuous Learning

One important feature of a cognitive system is **continuous improvement**.

New Data / User Feedback

↓

Model Update

↓

Learning

↓

Better Decision

The system uses new information and feedback to improve its knowledge and performance.

---

5. Example: Cognitive Healthcare System

Patient Data

↓

Data Processing

↓

Medical Knowledge Base

↓

NLP / Understanding

↓

Machine Learning

↓

Reasoning & Inference

↓

Diagnosis / Recommendation

↓

Doctor / Patient

↓

Feedback → Learning

For example, the system receives a patient's symptoms, understands the medical terms using NLP, compares them with its knowledge base, applies AI reasoning, and provides a possible diagnosis or recommendation to assist the doctor.

---

6. Applications of Cognitive Systems

Cognitive systems are used in:

- **Healthcare:** Diagnosis and decision support
- **Banking:** Fraud detection and risk analysis
- **Retail:** Recommendation systems
- **Education:** Intelligent tutoring
- **Customer Service:** Chatbots and virtual assistants
- **Manufacturing:** Predictive maintenance

Conclusion

A **Cognitive System** combines **data processing, knowledge representation, NLP, machine learning, reasoning, and decision-making** to create intelligent applications. It can learn from experience, understand information, handle uncertainty, and provide useful recommendations or actions.